tkz-grapheur [en]

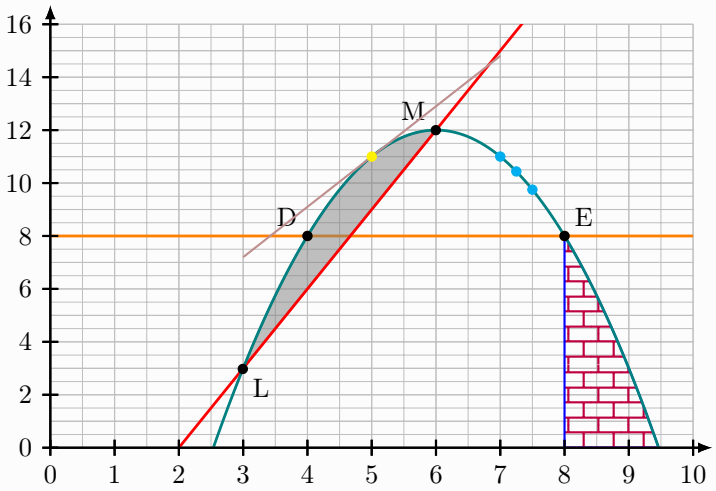
A grapher, based

on TikZ and xint.

Version 0.32b - 26/10/2026

Cédric Pierquet
c pierquet - at - outlook . fr
https://github.com/cpierquet/latex-packages/tree/main/tkz-grapheur

---





To my Dad.

Contents

1 Introduction **3**
- 1.1 Description and general ideas . . . 3
- 1.2 Overall operation . . . 3
- 1.3 Packages used, and package options . . . 3
- 1.4 Warnings . . . 4
- 1.5 Introductory example . . . 4

2 Basic Styles and Environment Creation **6**
- 2.1 Basic Styles . . . 6
- 2.2 Creating the environment . . . 6
  - 2.2.1 Manual values . . . 6
  - 2.2.2 With choice of dimensions . . . 8
- 2.3 Grids and axes . . . 9
- 2.4 Adding values manually . . . 11
- 2.5 Nodes for window or axis . . . 13

3 Specific definition commands **14**
- 3.1 Draw a line . . . 14
- 3.2 Define a function, draw the curve of a function . . . 15
- 3.3 Define/draw an interpolation curve (simple) . . . 16
- 3.4 Define/draw an interpolation curve (Hermite) . . . 17
- 3.5 Define points as nodes . . . 18
- 3.6 Mark Points . . . 20
- 3.7 Retrieve node coordinates . . . 21
- 3.8 Place text . . . 21

4 Specific commands for using curves **22**
- 4.1 Image placement . . . 22
- 4.2 Antecedent determination . . . 23
- 4.3 Antecedent construction . . . 24
- 4.4 Intersections of two curves . . . 26
- 4.5 Extrema . . . 27
- 4.6 Integrals (improved version) . . . 31
- 4.7 Tangents . . . 35
- 4.8 Linear inequality . . . 38

5 Commands specific to two-variable statistics **40**
- 5.1 Limitations . . . 40
- 5.2 The point scatter . . . 40
- 5.3 The regression line . . . 40
- 5.4 Other regressions . . . 41

6 Auxiliary commands **45**
- 6.1 Intro . . . 45
- 6.2 Formatted rounding . . . 45
- 6.3 Random number under constraints . . . 45
- 6.4 Monte-Carle method . . . 47
- 6.5 Some pgfplots macros . . . 49

7 History **51**

1 Introduction

1.1 Description and general ideas

With this modest package, far from the capabilities offered by the excellent packages `tkz-*`[1] (by Alain Matthes) or `tzplot`[2] (by In-Sung Cho), it is possible to work on function graphs, in Ti*k*Z language, in an *intuitive* and *explicit* way.

Concerning the overall operation:

- particular styles for the objects used have been defined, but they can be modified locally;
- the name of the commands is in *operational* form, so that the construction of the graphic elements has an almost *algorithmic* form.

1.2 Overall operation

To schematize, it *is enough*:

- to declare the parameters of the graphics window (**units in cm !**);
- to display grid/axes/graduations;
- to declare functions or interpolation curves;
- to possibly declare particular points;
- to place a point scatter.

It will then be possible:

- to draw curves;
- to graphically determine images or backgrounds;
- to add elements of derivation (tangents) or integration (domain);
- to draw a linear fit line or the curve of another fit.

1.3 Packages used, and package options

The package uses:

- `tikz`, with the libraries `calc,intersections,patterns,patterns.meta,bbox`;
- `simplekv`, `xintexpr`, `xstring`, `listofitems`;
- `xint-regression`[3] (for regressions, switchable via `[noxintreg]`).

The package also loads `siunitx` with the classic options, but it is possible not to load it using the `[nosiunitx]` option.

The package also loads the Ti*k*Z `babel` library, but it is possible not to load it using the `[notikzbabel]` option.

The different options are obviously cumulative.

[1] for example tkz-base https://ctan.org/pkg/tkz-base and tkz- fct https://ctan.org/pkg/tkz-fct.
[2] CTAN: https://ctan.org/pkg/tzplot.
[3] CTAN: https://ctan.org/pkg/xint-regression.

```
%loading by default, with french setup of siunitx
\usepackage{tkz-grapheur}
%loading by default, with normal setup of siunitx
\usepackage[english]{tkz-grapheur}

%loading without sinuitx, to be loaded manually
\usepackage[nosiunitx]{tkz-grapheur}

%loading without tikz.babel
\usepackage[notikzbabel]{tkz-grapheur}
```

Also note that certain commands can use packages like `nicefrac`, which will therefore have to be loaded if necessary.

Concerning the *calculations* and *plots* part, the `xint` package takes care of it.

1.4 Warnings

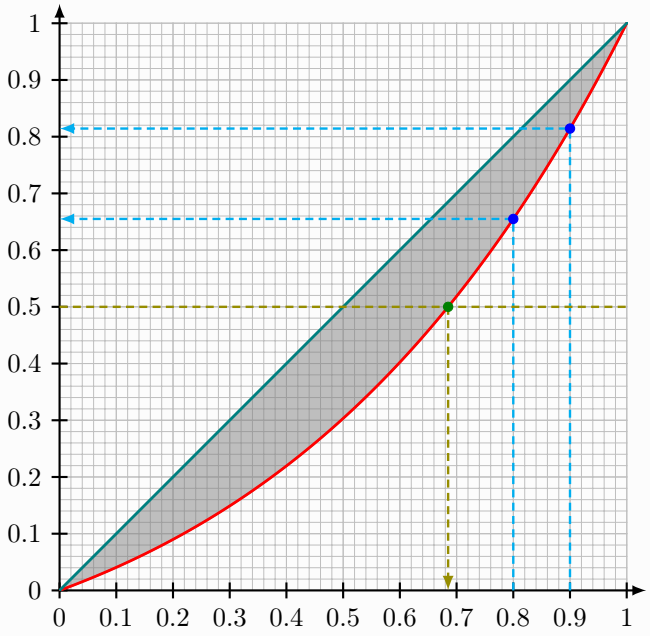
It is possible, due to the (multiple) calculations carried out internally, that the compilation time may be a little *long*.

The precision of the (determination) results seems to be around 10^{-4}, which should normally guarantee *satisfactory* plots and readings. It is still advisable to be cautious about the results obtained and those expected.

1.5 Introductory example

For example, we can start from the following example to *illustrate* the flow of the commands for this package. The commands and syntax will be detailed in the following sections!

```
\begin{GraphTikz}%
  [x=7.5cm,y=7.5cm,Xmin=0,Xmax=1.001,Xgrid=0.1,Xgrids=0.02,
  Ymin=0,Ymax=1.001,Ygrid=0.1,Ygrids=0.02]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]%
    {0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1}
    {0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1}
  \DefineCurve[Name=cf,Start=0,End=1]<f>{x*exp(x-1)}
  \DefineCurve[Name=delta,Start=0,End=1]<D>{x}
  \DrawIntegral[Type=fct/fct]{f(x)}[D(x)]{0}{1}
  \DrawCurve[Color=red]{f(x)}
  \DrawCurve[Color=teal]{D(x)}
  \DrawRanges[Colors=blue/cyan,Lines]{f}{0.8,0.9}
  \DrawCounterimage[Colors=green!50!black/olive,Lines]{cf}{0.5}
\end{GraphTikz}
```


1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1

2 Basic Styles and Environment Creation

2.1 Basic Styles

The styles used for plots are given below.

For *simplicity* purposes, only the color of the elements can be configured, but if the user wishes, he can redefine the proposed styles.

```
%parameters declared and stored (usable in the environment a posteriori)
\tikzset{
  Xmin/.store in=\pflxmin,Xmin/.default=-3,Xmin=-3,
  Xmax/.store in=\pflxmax,Xmax/.default=3,Xmax=3,
  Ymin/.store in=\pflymin,Ymin/.default=-3,Ymin=-3,
  Ymax/.store in=\pflymax,Ymax/.default=3,Ymax=3,
  Origx/.store in=\pflOx,Origx/.default=0,Origx=0,
  Origy/.store in=\pflOy,Origy/.default=0,Origy=0,
  Xgrid/.store in=\pflgrillex,Xgrid/.default=1,Xgrid=1,
  Xgrids/.store in=\pflgrillexs,Xgrids/.default=0.5,Xgrids=0.5,
  Ygrid/.store in=\pflgrilley,Ygrid/.default=1,Ygrid=1,
  Ygrids/.store in=\pflgrilleys,Ygrids/.default=0.5,Ygrids=0.5
}
```

We therefore find:

- the origin of the mark (`Origx`/`Origy`);
- the extreme values of the axes (`Xmin`/`Xmax`/`Ymin`/`Ymax`);
- the parameters of the main and secondary grids (`Xgrid`/`Xgrids`/`Ygrid`/`Ygrids`).

Concerning the styles of *objects*, they are given below.

```
\tikzset{tkzgrphnode/.style={}}
\tikzset{tkzgrphpoint/.style={line width=0.95pt}}
\tikzset{tkzgrphpointc/.style={radius=1.75pt}}
\tikzset{tkzgrphscatter/.style={radius=1.75pt}}
\tikzset{tkzgrphframe/.style={line width=0.8pt,gray}}
\tikzset{tkzgrphcurve/.style={line width=1.05pt}}
\tikzset{tkzgrphline/.style={line width=0.8pt}}
\tikzset{tkzgrpharrowl/.style={<-,>=latex}}
\tikzset{tkzgrpharrowr/.style={->,>=latex}}
\tikzset{tkzgrpharrowlr/.style={<->,>=latex}}
\tikzset{tkzgrphcounterimage/.style={line width=0.9pt,densely dashed}}
\tikzset{tkzgrphrange/.style={line width=0.9pt,densely dashed,->,>=latex}}
\tikzset{tkzgrphgridp/.style={thin,lightgray}}
\tikzset{tkzgrphgrids/.style={very thin,lightgray}}
\tikzset{tkzgrphaxes/.style={line width=0.8pt,->,>=latex}}
```

The idea is therefore to be able to redefine styles globally or locally, and possibly add elements, using `mystyle/.append style={...}`.

2.2 Creating the environment

2.2.1 Manual values

The proposed environment is based on Ti*k*Z, so that any *classic* command linked to Ti*k*Z can be used alongside the package commands!

```
\begin{GraphTikz}[tikz options]<keys>
  %code(s)
\end{GraphTikz}
```

The `[tikz options]` are the *classic* options that can be passed to a Ti*k*Z environment, as well as the axes/grids/window keys presented previously.

The specific (and optional) `<keys>` are:

- `ThickGrad`: size of the axis graduations (`3pt` for 3pt *above* and 3pt *below*);
- `Frame`: boolean (`false` by default) to display a frame which delimits the graphic window (excluding possible graduations).

```
\begin{GraphTikz}
  [x=0.075cm,y=0.03cm,Xmin=0,Xmax=160,Xgrid=20,Xgrids=10,
  Origy=250,Ymin=250,Ymax=400,Ygrid=25,Ygrids=5]
  <Frame>
\end{GraphTikz}
```



```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  <Frame>
\end{GraphTikz}
```

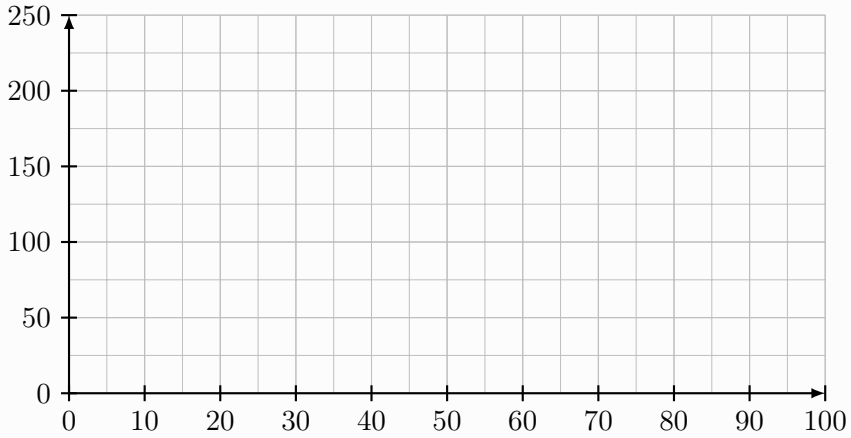


It will obviously be more meaningful with the added graphic elements!

2.2.2 With choice of dimensions

It is also possible (currently being tested) to specify the dimensions of the graph, letting the code determine the correct units.
In this case, the keys and arguments differ slightly, notably via the key `<Size=width/height>`:

```
%fixed dimensions (excluding graduations)
\begin{GraphTikz}%
  [Xgrid=10,Xgrids=5,Ygrid=50,Ygrids=25]
  %Xgrid(s) and/or Ygrid(s) in this case
  <Xmin=0,Xmax=100,Ymin=0,Ymax=250,Size={10cm/5cm}>
  \DrawAxisGrids[]{auto}{auto}
\end{GraphTikz}
```



2.3 Grids and axes

The first command *useful* will allow you to create the grids, axes and graduations.

```
%in the GraphiqueTikz environment
\DrawAxisGrids[keys]{gradX}{gradY}
```

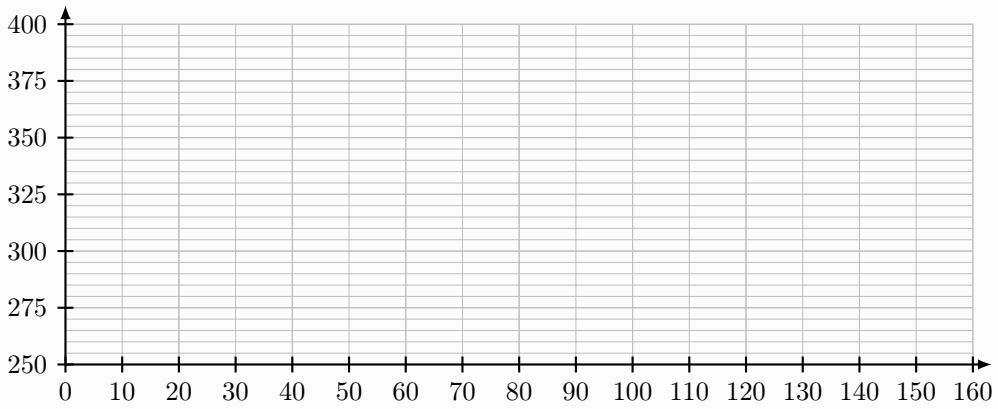
The optional `[keys]` available are:

- `Grid`: boolean (`true` by default) to display the grids (for a single grid, simply set the identical parameters for `Xgrid`/`Xgrids` or `Ygrid`/`Ygrids`);
- `Enlarge`: addition at the end of the axes (`0` by default);
- `Grads`: boolean (`true` by default) for graduations;
- `Font`: global font for graduations `empty` by default;
- `Format`: special formatting (see below) of the axis values.

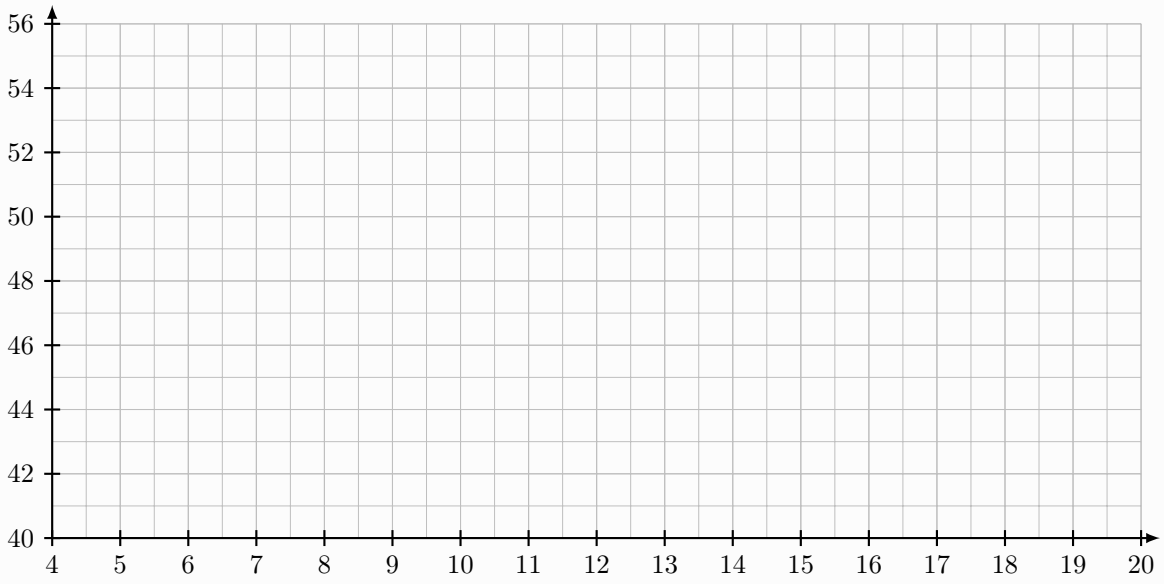
Concerning the `Format` key, it allows you to specify a specific setting for the axis values.
It can be given in the form `fmt` for combined formatting, or in the form `fmtX/fmtY` to differentiate the formatting.
The possible options are:

- `num`: format with siunitx;
- `year`: format in year;
- `frac`: format as fraction frac;
- `dfrac`: format as fraction dfrac;
- `nfrac`: format as fraction nicefrac; (to load!)
- `trig`: format in trig with frac;
- `dtrig`: format in trig with dfrac;
- `ntrig`: format in trig with nfrac;
- `sqrt`: format in root with frac;
- `dsqrt`: format in root with dfrac;
- `nsqrt`: format in root with nicefrac.

```
\begin{GraphTikz}
  [x=0.075cm,y=0.03cm,Xmin=0,Xmax=160,Xgrid=20,Xgrids=10,
  Origy=250,Ymin=250,Ymax=400,Ygrid=25,Ygrids=5]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{0,10,...,160}{250,275,...,400}
\end{GraphTikz}
```

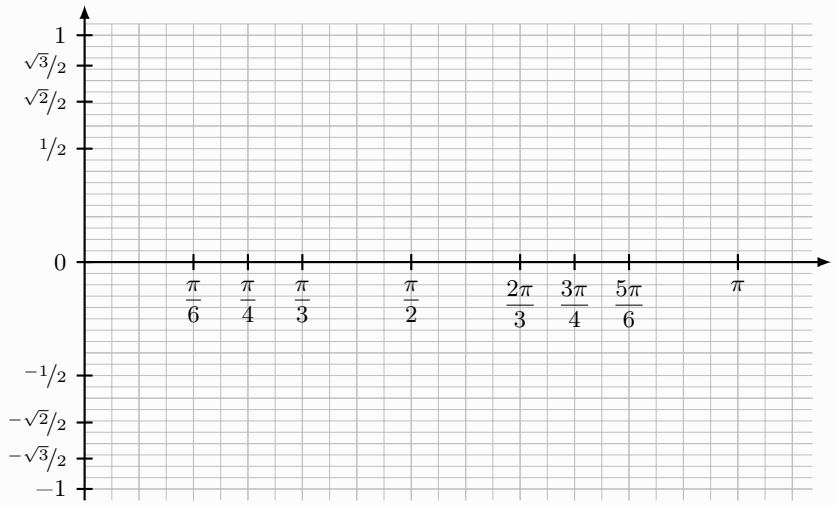



```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
\end{GraphTikz}
```



Note that there are the Boolean keys `[Behind]` (without the graduations) and `[Above]` (without the grid) to display the axes in *under/over*-printing mode in the case of integrals for example.

```
\begin{GraphTikz}%
  [x=2.75cm,y=3cm,
  Xmin=0,Xmax=3.5,Xgrid=pi/12,Xgrids=pi/24,
  Ymin=-1.05,Ymax=1.05,Ygrid=0.2,Ygrids=0.05]
  \DrawAxisGrids[Enlarge=2.5mm,Format=dtrig/nsqrt,Font=\footnotesize]%
  {pi/6,pi/4,pi/3,pi/2,2*pi/3,3*pi/4,5*pi/6,pi}
  {0,sqrt(2)/2,1/2,sqrt(3)/2,1,-1,-sqrt(3)/2,-1/2,-sqrt(2)/2}
\end{GraphTikz}
```



In the case where the formatting does not give satisfactory result(s), it is possible to use a generic command for placing the graduations.

2.4 Adding values manually

It is also possible to use a specific command to place values on the axes, independently of an *automated* formatting system.

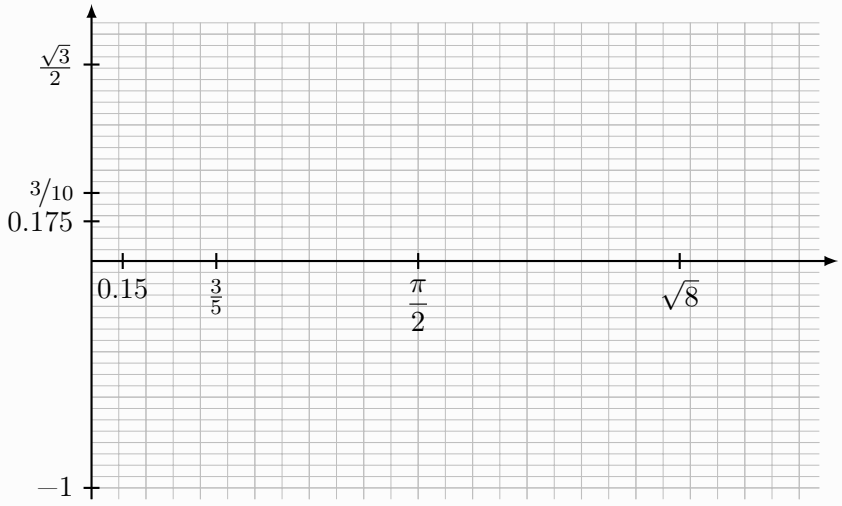
```
in the environment
\AddXvalues[keys]{grads}{formatted values}
\AddYvalues[keys]{grads}{formatted values}
```

The optional `[keys]` available are:

- `Font`: global font for graduations `empty` by default;
- `Lines`: boolean to add the tick marks `true` by default.

The mandatory arguments correspond to the x-coordinates (in Ti*k*Z language) and to the labels (in LaTeX language) of the graduations.

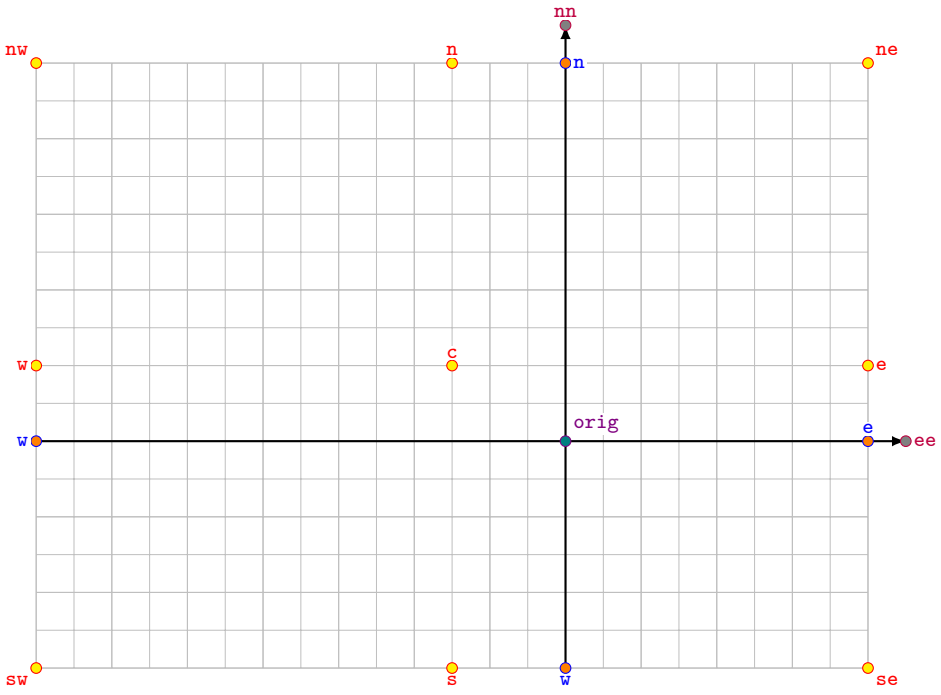
```
\begin{GraphTikz}%
  [x=2.75cm,y=3cm,
  Xmin=0,Xmax=3.5,Xgrid=pi/12,Xgrids=pi/24,
  Ymin=-1.05,Ymax=1.05,Ygrid=0.2,Ygrids=0.05]
  \DrawAxisGrids[Grads=false,Enlarge=2.5mm]{}{}
  \AddXvalues
    {0.15,0.6,pi/2,2.8284}
    {\num{0.15},$\frac35$,$\displaystyle\frac{\pi}{2}$,$\sqrt{8}$}
  \AddYvalues
    {-1,0.175,0.3,sqrt(3)/2}
    {\num{-1},\num{0.175},$\nicefrac{3}{10}$,$\frac{\sqrt{3}}{2}$}
\end{GraphTikz}
```



2.5 Nodes for window or axis

Alongside the creation of the window (and possibly the axes), nodes are created for later reuse (e.g., text placement).

- 9 nodes for window:
  - `(graph-nw)`, `(graph-n)`, `(graph-ne)` ;
  - `(graph-w)`, `(graph-c)`, `(graph-e)` ;
  - `(graph-sw)`, `(graph-s)`, `(graph-se)` ;
- 4 nodes for axis:
  - `(xaxis-w)`, `(xaxis-e)` ;
  - `(yaxis-s)`, `(yaxis-n)` ;
- 1 node for origin:
  - `(axis-orig)` ;
- 2 nodes for *enlarge* axis:
  - `(xaxis-ee)` ;
  - `(yaxis-nn)`.



3 Specific definition commands

3.1 Draw a line

The idea is to propose a command to draw a line (or an asymptote), from:

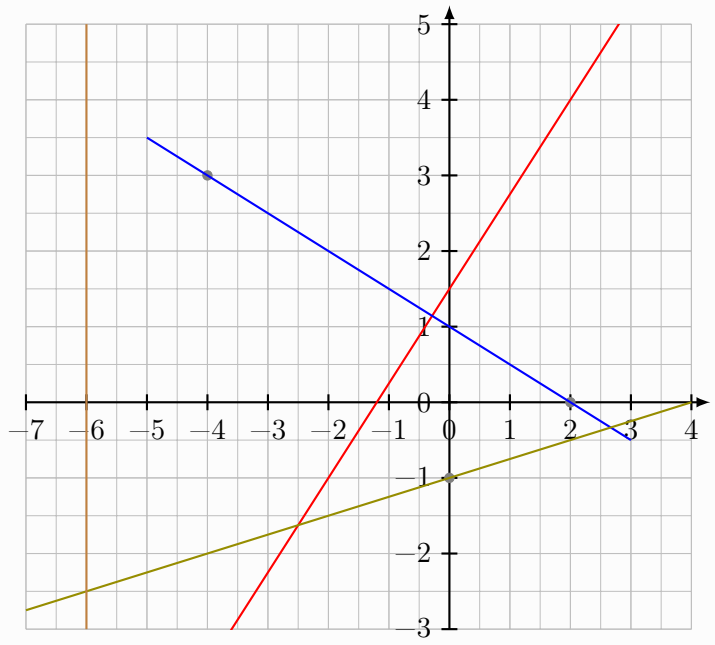
- of two points (or nodes);
- of a point (or node) and the slope.

```
%in the GraphiqueTikz environment
\DrawLine[keys]{point or node}{point or node or slope}
\DrawAsymptote[keys]{x value}
```

The optional `[keys]` available are:

- `Name`: possible name of the plot (for reuse);
- `Slope`: boolean to specify that the slope is used (`false` by default);
- `Start`: start of the plot (`\pflxmin` by default);
- `End`: end of the plot (`\pflxmax` by default);
- `Color`: color of the trace (`black` by default).

```
\begin{GraphTikz}%
  [x=0.8cm,y=1cm,Xmin=-7,Xmax=4,Ymin=-3,Ymax=5]
  \DrawAxisGrids[Enlarge=2.5mm]{auto}{auto}
  \DefinePts[Mark,Color=gray]{A/-4/3,B/2/0,C/0/-1}
  \DrawLine[Color=red]{(-2,-1)}{(2,4)}
  \DrawLine[Color=blue,Start=-5,End=3]{(A)}{(B)}
  \DrawLine[Color=olive,Slope]{(C)}{0.25}
  \DrawAsymptote[Color=brown]{-6}
\end{GraphTikz}
```



3.2 Define a function, draw the curve of a function

The idea is to define a function, for later reuse. This command *creates* the function, without tracing it, because in certain cases elements will have to be traced beforehand.

There is also a command to plot the curve of a previously defined function.

```
%in the GraphiqueTikz environment
\DefineCurve[keys]<fct name>{xint formula}
\DrawCurve[keys]{xint formula}
```

The optional `[keys]` for definition or tracing are:

- `Start`: lower bound of the definition set (`\pflxmin` by default);
- `End`: lower bound of the definition set (`\pflxmax` by default);
- `Name`: name of the curve (important for the rest!);
- `Color`: color of the trace (`black` by default);
- `Step`: plot step (it is determined *automatically* at the start but can be modified);
- `Trace`: boolean to also trace the curve (`false` by default).

```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
  %definition of the function + drawing of the curve
  \DefineCurve[Name=cf,Start=5,End=19]<f>{-2*x+3+24*log(2*x)}
  \DrawCurve[Color=red,Start=5,End=19]{f(x)}
  %or in a single command if "sufficient"
  %\DefineCurve[Name=cf,Start=5,End=19,Trace]<f>{-2*x+3+24*log(2*x)}
\end{GraphTikz}
```



3.3 Define/draw an interpolation curve (simple)

It is also possible to define a curve via support points, therefore a simple interpolation curve.

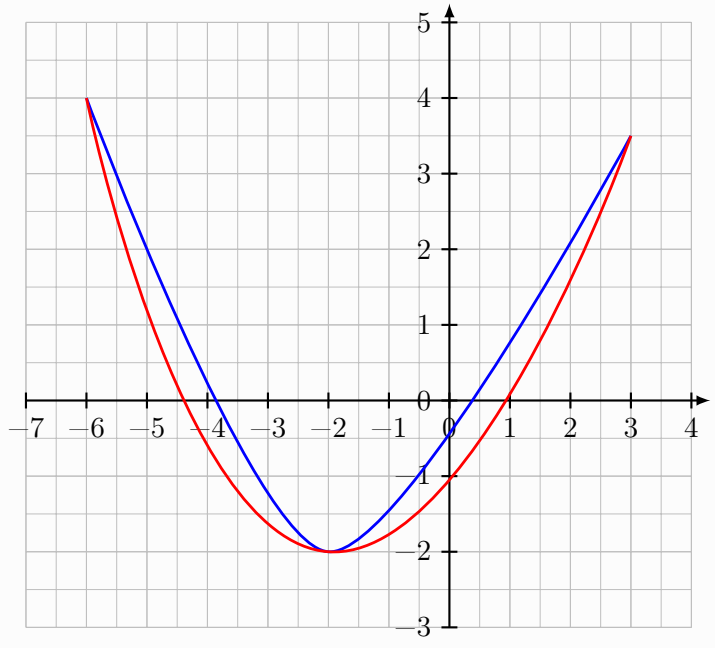
```
%in the GraphiqueTikz environment
\DefineInterpoCurve[keys]{list of support points}
\DrawInterpoCurve[keys]{list of support points}
```

The optional `[keys]` for definition or tracing are:

- `Name`: name of the interpolation curve (important for the rest!);
- `Color`: color of the trace (`black` by default);
- `Tension`: setting the *tension* of the interpolation plot (`0.5` by default);
- `Trace`: boolean to also trace the curve (`false` by default).

The mandatory argument allows you to specify the list of support points in the form `(x1,y1)(x2,y2)...`.

```
\begin{GraphTikz}%
  [x=0.8cm,y=1cm,Xmin=-7,Xmax=4,Ymin=-3,Ymax=5]
  \DrawAxisGrids[Enlarge=2.5mm]{-7,-6,...,4}{-3,-2,...,5}
  %simple interpolation curves (with diff tension)
  \DefineInterpoCurve[Name=interpotest,Color=blue,Trace]%
    {(-6,4)(-2,-2)(3,3.5)}
  \DefineInterpoCurve[Name=interpotest,Color=red,Trace,Tension=1]%
    {(-6,4)(-2,-2)(3,3.5)}
\end{GraphTikz}
```



3.4 Define/draw an interpolation curve (Hermite)

It is also possible to define a curve via support points, therefore an interpolation curve with derivative control.

Some operations require different techniques depending on the type of function used, a Boolean key `Spline` will allow the code to adapt its calculations depending on the object used.

```
%in the GraphiqueTikz environment
\DefineSplineCurve[keys]{list of support points}[\macronomspline]
\DrawSplineCurve[keys]{list of support points}[\macronomspline]
```

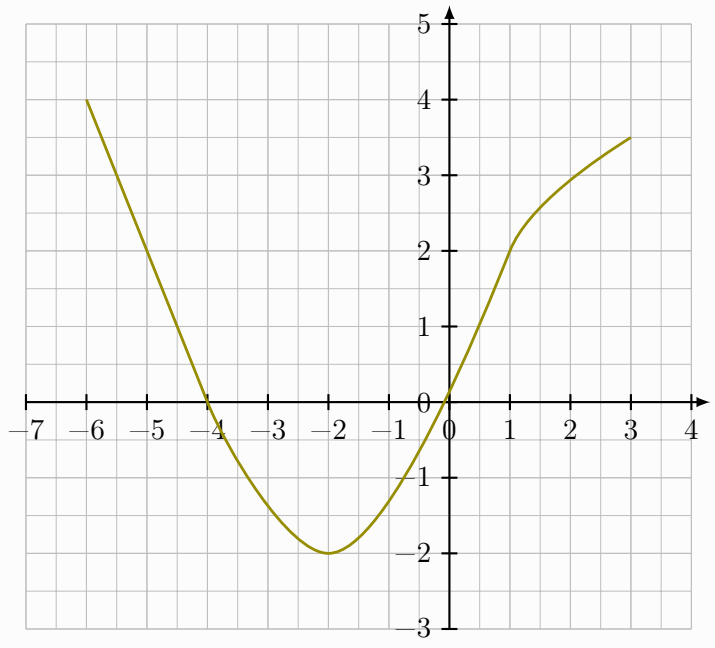
The optional `[keys]` for definition or tracing are:

- `Name`: name of the interpolation curve (important for the rest!);
- `Coeffs`: modify (see the ProfLycee[4] the *coefficients* of the spline;
- `Color`: color of the trace (`black` by default);
- `Trace`: boolean to also trace the curve (`false` by default).

The mandatory argument allows you to specify the list of support points in the form `x1/y1/f'1§x2/y2/f'2§...` with:

- `xi/yi` the coordinates of the point;
- `f'i` the derivative at the support point.

```
\begin{GraphTikz}%
  [x=0.8cm,y=1cm,Xmin=-7,Xmax=4,Ymin=-3,Ymax=5]
  \DrawAxisGrids[Enlarge=2.5mm]{-7,-6,...,4}{-3,-2,...,5}
  %definition of the list of spline support points
  \def\LISTETEST{-6/4/-2§-5/2/-2§-4/0/-2§-2/-2/0§1/2/2§3/3.5/0.5}
  %definition and plot of the cubic spline
  \DefineSplineCurve[Name=splinetest,Trace,Color=olive]{\LISTETEST}
\end{GraphTikz}
```



[4] CTAN documentation: https://ctan.org/pkg/proflycee

3.5 Define points as nodes

The second idea is to work with Ti*k*Znodes, which could be useful for tangent plots, representations of integrals...

It is also possible to define nodes for *image* points.

Certain commands (explained later) allow you to determine particular points of curves in the form of nodes, so it seems interesting to be able to define them directly.

```
%by coordinates
\DefinePts[keys]{Name1/x1/y1,Name2/x2/y2,...}
```

The optional `[keys]` available are:

- `Mark`: boolean to mark points (`false` by default);
- `Color`: color of the points, if `Mark=true` (`black` by default).

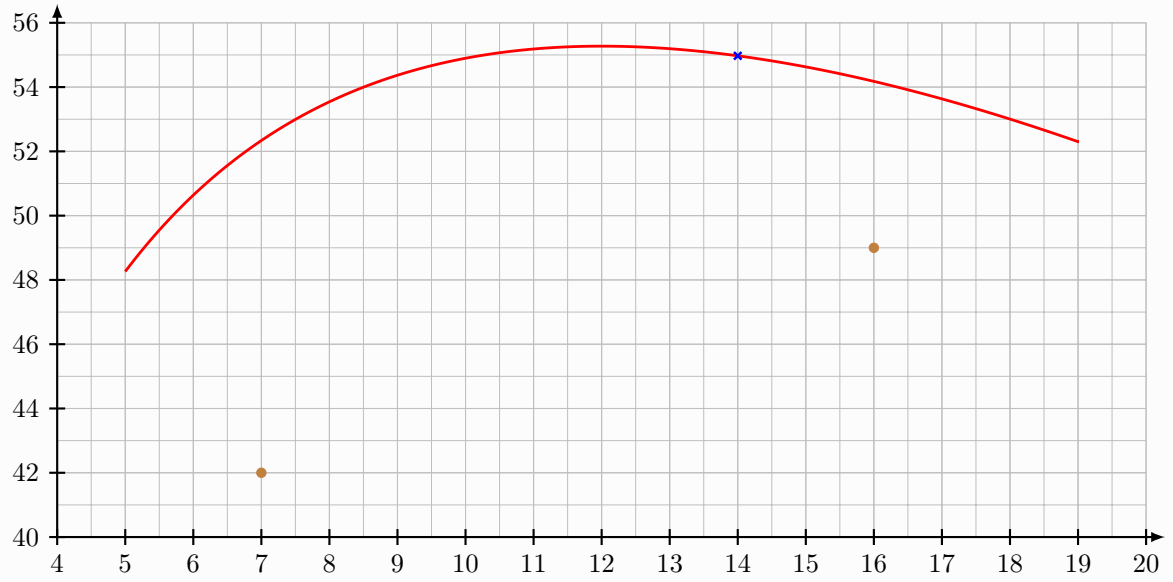
```
%as image
\DefineRange[keys]{object}{abscissa}
```

The optional `[keys]` available are:

- `Name`: node name (`empty` by default);
- `Spline`: boolean to specify that a spline is used (`false` by default).

The first mandatory argument is the *object* considered (name of the curve for the spline, function otherwise); the second is the abscissa of the point considered.

```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
  %definition of the function + drawing of the curve
  \DefineFunction[Name=cf,Start=5,End=19,Trace,Color=red]<f>{-2*x+3+24*log(2*x)}
  %manual nodes
  \DefinePts[Mark,Color=brown]{A/7/42,B/16/49}
  %imagenode
  \DefineRange[Name=IMGf]{f}{14}
  \MarkPts*[Style=x,Color=blue]{(IMGf)} %see next section;-)
\end{GraphTikz}
```



3.6 Mark Points

The idea is to offer something to score points with a particular style.

```
%in the GraphiqueTikz environment
\MarkPts(*)[keys]<font>{list}
```

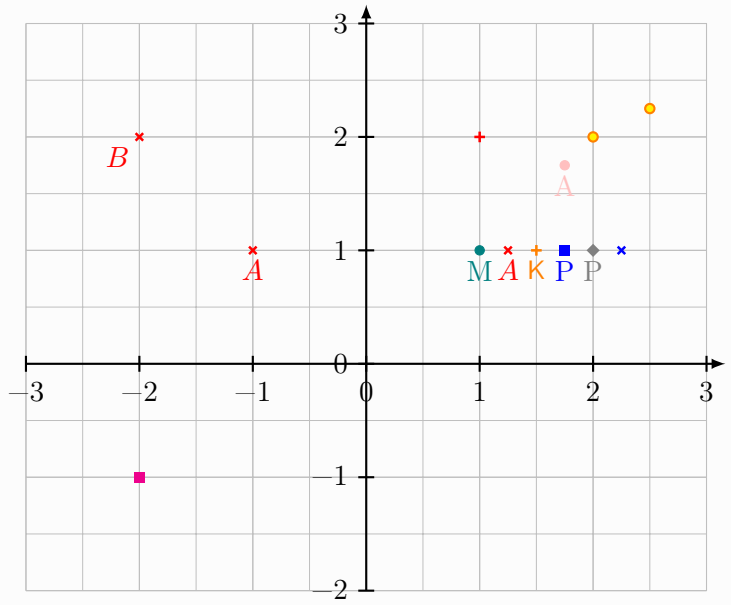
The *starred* version scores the points without the *names*, while the *unstarred* version displays them:

- in the case of the *starred* version, the list should be given in the form `(ptA),(ptB),...`;
- otherwise, the list should be given in the form `(ptA)/poslabelA/labelA,...`.

The optional `[keys]` available are:

- `Color`: color (`black` by default);
- `Style`: style of marks (`o` by default).

```
\begin{GraphTikz}[x=1.5cm,y=1.5cm,Ymin=-2]
  \DrawAxisGrids[Enlarge=2.5mm]{auto}{auto}
  \DefinePts{A/1.75,-1.25}\MarkPts[Color=pink]{(A)/below/A} %round (default)
  \MarkPts[Color=teal]{(1,1)/below/M}
  \MarkPts[Color=red,Style=x]{(1.25,1)/below/$A$} %cross
  \MarkPts[Color=orange,Style=+]<\small\sffamily>{(1.5,1)/below/K} %plus
  \MarkPts[Color=blue,Style=c]{(1.75,1)/below/P} %square
  \MarkPts[Color=gray,Style=d]{(2,1)/below/P} %diamond
  \MarkPts*[Color=orange/yellow]{(2,2),(2.5,2.25)} %two-tone round
  \MarkPts*[Style=+,Color=red]{(1,2)}
  \MarkPts*[Style=x,Color=blue]{(2.25,1)}
  \MarkPts*[Style=c,Color=magenta]{(-2,-1)}
  \MarkPts[Color=red,Style=x]{(-1,1)/below/$A$,(-2,2)/below left/$B$}
\end{GraphTikz}
```

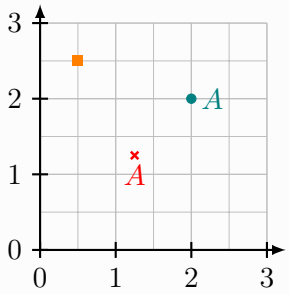


Note that it is also possible to modify the size of the `o/x/+/c` marks via the `[keys]`:

- `Sizex=...` (`2pt` by default) for points *cross*;
- `Sizeo=...` (`1.75pt` by default) for the points *circle*;

- `Sizec=...` (`2pt` by default) for the *square* points.

```
\begin{GraphTikz}[x=1cm,y=1cm,Xmin=0,Ymin=0]
  \DrawAxisGrids[Enlarge=2.5mm]{auto}{auto}
  \MarkPts[Color=red,Style=x,Size=3.5pt]{(1.25,1.25)/below/$A$}
  \MarkPts[Color=teal,Size=2.5pt]{(2,2)/right/$A$}
  \MarkPts*[Color=orange,Style=c,Size=4pt]{(0.5,2.5)}
\end{GraphTikz}
```



3.7 Retrieve node coordinates

It is also possible, with a view to reusing coordinates, to recover the coordinates of a node (defined or determined).

The calculations are carried out by floating according to the (re)calculated units, the values are therefore approximated !

```
%in the GraphiqueTikz environment
\GetXcoord{node}[\macrox]
\GetYcoord{node}[\macroy]
\GetXYcoord{node}[\macrox][\macroy]
```

3.8 Place text

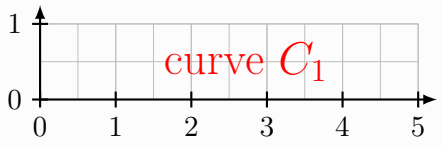
Note that a text placement command is available.

```
%in the GraphiqueTikz environment
\DrawTxt[keys]{(node or coordinates)}{text}
```

The available `[keys]` are:

- `Font=...` (`\normalsize\normalfont` by default) for the font;
- `Color=...` (`black` by default) for the color;
- `Position=...` (`empty` by default) for the position of the text relative to the coordinates.

```
\begin{GraphTikz}[x=1cm,y=1cm,Xmin=0,Xmax=5,Ymin=0,Ymax=1]
  \DrawAxisGrids[Enlarge=2.5mm]{auto}{auto}
  \DrawTxt[Color=red,Font=\LARGE,Position=right]{(1.5,0.5)}{curve $C_1$}
\end{GraphTikz}
```



4 Specific commands for using curves

4.1 Image placement

It is possible to place points (images) on a curve, with possible construction lines.
The function/curve used must have been declared previously for this command to work.

```
%in the GraphiqueTikz environment
\DrawRanges[keys]{function or curve}{list of abscissa}
```

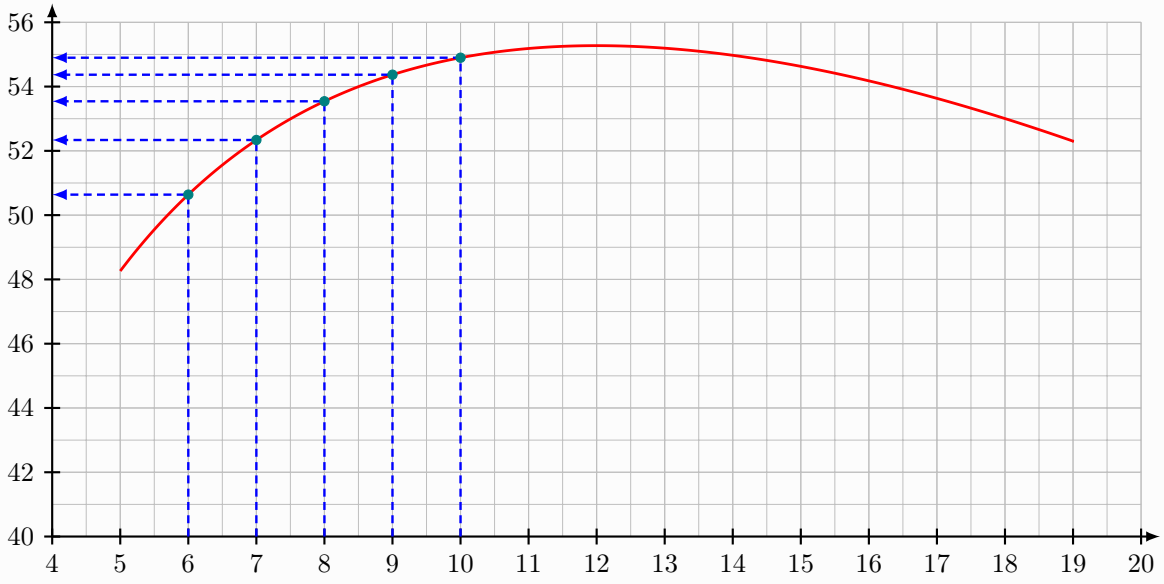
The optional `[keys]` available are:

- `Lines`: boolean to display construction traits (`false` by default);
- `Colors`: color of the points/lines, in the form `Couleurs` or `ColPoint/ColLines`;
- `Spline`: boolean to specify that the curve used is defined as a spline (`false` by default).

The first mandatory argument allows you to specify:

- the name of the curve in the case `Spline=true`;
- the name of the function otherwise.

```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
  %definition of the function + drawing of the curve
  \DefineCurve[Name=cf,Start=5,End=19,Trace,Color=red]<f>{-2*x+3+24*log(2*x)}
  %images
  \DrawRanges[Lines,Colors=teal/blue]{f}{6,7,8,9,10}
\end{GraphTikz}
```



4.2 Antecedent determination

It is possible to graphically determine the antecedents of a given reality.
The function/curve used must have been declared previously for this command to work.

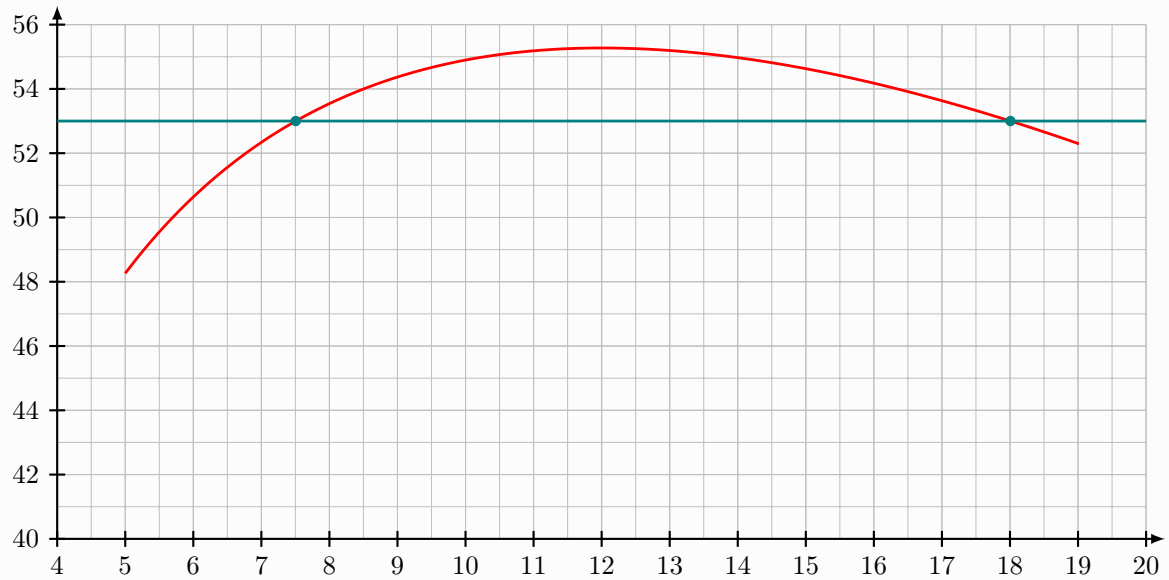
```
%in the GraphiqueTikz environment
\FindCounterimage[keys]{curve}{k}
```

The optional `[keys]` available are:

- `Name`: base of the name of the **nodes** intersection (`S` by default, which will give S-1, S-2, etc);
- `Disp`: boolean to display the points (`true` by default);
- `Color`: color of the points (`black` by default);
- `DispLine`: boolean to display the horizontal line (`false` by default).

The first mandatory argument allows you to specify the **name** of the curve.
The second mandatory argument allows you to specify the value to reach.

```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
  %definition of the function + drawing of the curve
  \DefineCurve[Name=cf,Start=5,End=19,Trace,Color=red]<f>{-2*x+3+24*log(2*x)}
  %history
  \FindCounterimage[Color=teal,DispLine,Disp]{cf}{53}
  %the two antecedents are at nodes (S-1) and (S-2)
\end{GraphTikz}
```



4.3 Antecedent construction

It is possible to graphically construct the antecedents.
The function/curve used must have been declared previously for this command to work.

```
%in the GraphiqueTikz environment
\DrawCounterimage[keys]{curve}{k}
```

The optional `[keys]` available are:

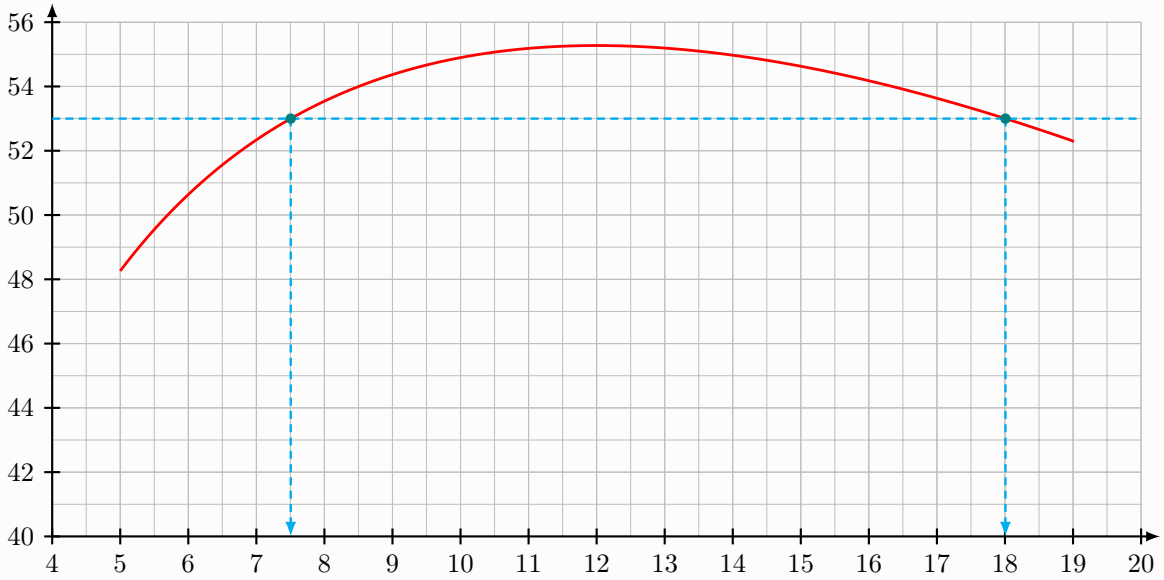
- `Colors`: color of the points/lines, in the form `Color` or `ColPoint/ColLines`;
- `Name`: name *possible* for the intersection points linked to the antecedents (`empty` by default);
- `Lines`: boolean to display construction traits (`false` by default).

The first mandatory argument allows you to specify the **name** of the curve.
The second mandatory argument allows you to specify the value to reach.

```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
  %definition of the function + drawing of the curve
  \DefineCurve[Name=cf,Start=5,End=19,Trace,Color=red]<f>{-2*x+3+24*log(2*x)}
  %history
  \DrawCounterimage[Colors=teal/cyan,Lines,Name=PO]{cf}{53}
  \GetXcoord{(PO-1)}[\premsol]
  \GetXcoord{(PO-2)}[\deuxsol]
\end{GraphTikz}

Graphically, the antecedents of 53 are (approximately):

\begin{itemize}
  \item \num{\premsol}
  \item \num{\deuxsol}
\end{itemize}
```



Graphically, the antecedents of 53 are (approximately):

- 7.505 389 008 644 637
- 18.007 022 378 275 07

4.4 Intersections of two curves

It is also possible to determine (in the form of nodes) the possible points of intersection of two previously defined curves.

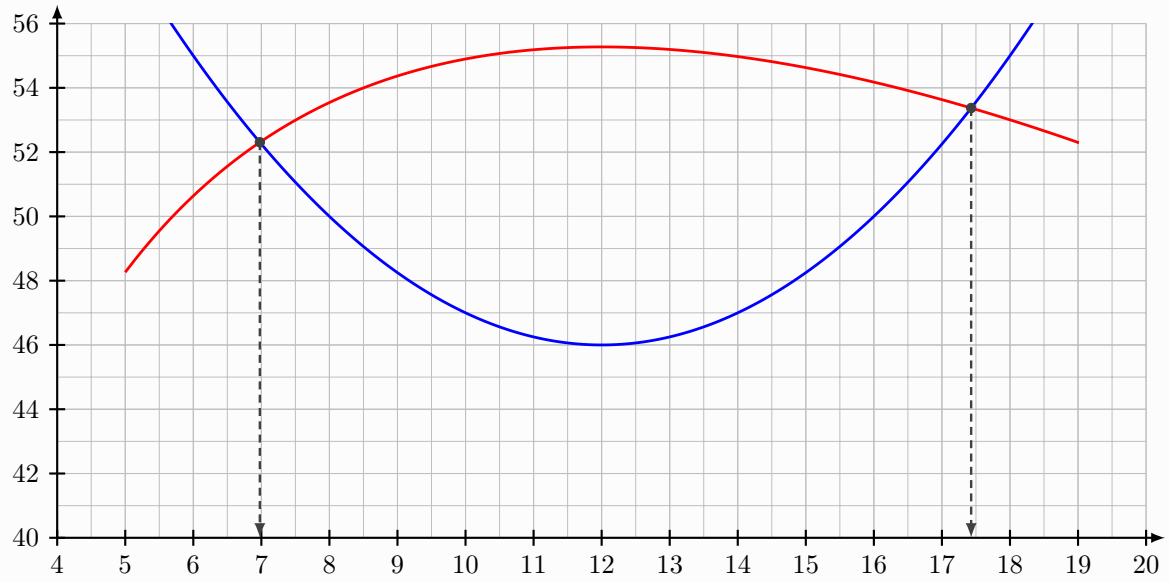
```
%in the GraphiqueTikz environment
\FindIntersections[keys]{curve1}{curve2}
```

The optional `[keys]` available are:

- `Name`: base of the name of the **nodes** intersection (`S` by default, which will give S-1, S-2, etc);
- `Disp`: boolean to display the points (`true` by default);
- `Color`: color of the points (`black` by default).

The first mandatory argument allows you to specify the **name** of the first curve.
The first mandatory argument allows you to specify the **name** of the second curve.

```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
  \DefineCurve[Name=cf,Start=5,End=19,Trace,Color=red]<f>{-2*x+3+24*log(2*x)}
  \DefineCurve[Name=cg,Start=5,End=19,Trace,Color=blue]<g>{0.25*(x-12)^2+46}
  %intersections, named (TT-1) and (TT-2)
  \FindIntersections[Name=TT,Color=darkgray,Display,Lines]{cf}{cg}
  %recovery of intersection points
  \GetXYcoord{(TT-1)}[\alphaA][\betaA]
  \GetXYcoord{(TT-2)}[\alphaB][\betaB]
\end{GraphTikz}\\
The solutions of $f(x)=g(x)$ are $\alpha \approx \num{\alphaA}$ and
$\beta \approx \num{\alphaB}$.\\
The points of intersection of the curves of $f$ and $g$ are therefore
$(\RoundNb[2]{\alphaA};\RoundNb[2]{\betaA})$ and
$(\RoundNb[2]{\alphaB};\RoundNb[2]{\betaB})$.
```



The solutions of $f(x) = g(x)$ are $\alpha \approx 6.977\,766\,172\,581\,613$ and $\beta \approx 17.429\,687\,326\,385\,03$.
The points of intersection of the curves of f and g are therefore $(6.98; 52.31)$ and $(17.43; 53.37)$.

4.5 Extrema

The idea (still *experimental*) is to offer commands to extract the extrema of a curve defined by the package.
The command creates the corresponding node, and it is therefore possible to retrieve its coordinates for later use.

It is possible, by specifying it, to work on the different curves managed by the package (function, interpolation, spline).
For singular curves, it is possible that the results are not quite those expected...

💣 For the moment, the *limitations* are:

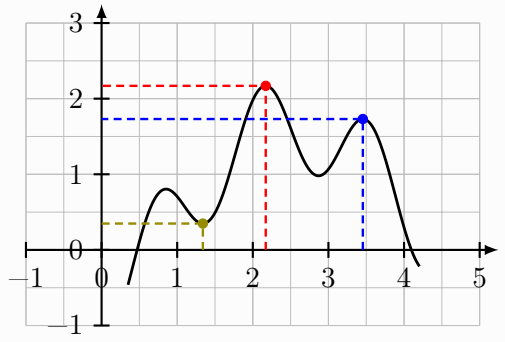
- no management of multiple extrema (only the first will be processed)...
- no management of extrema at the boundaries of the route...
- no automatic recovery of curve definition parameters...
- compilation time may be longer...

```
%in the GraphiqueTikz environment
\FindMax[keys]{object}[node created]
\FindMin[keys]{object}[node created]
```

The optional `[keys]` available are:

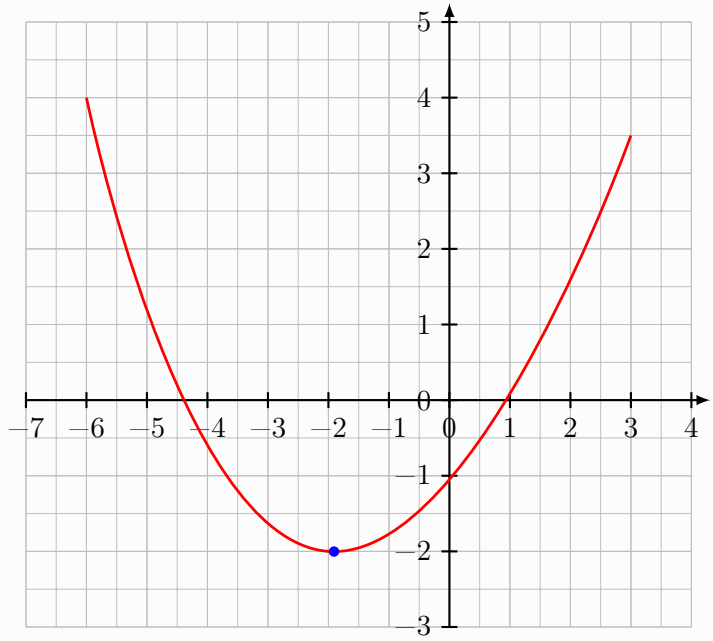
- `Method`: method, among `function/interpo/spline` for calculations (`function` by default);
- `Start`: start of the plot (`\pflxmin` by default);
- `End`: end of the plot (`\pflxmax` by default);
- `Step`: not in the plot if `function` (it is determined *automatically* at the start but can be modified);
- `Coeffs`: modify the *coefficients* of the spline if `spline`;
- `Tension`: setting the *tension* of the interpolation plot if `interpo`(`0.5` by default).

```
\begin{GraphTikz}[x=1cm,y=1cm,Xmin=-1,Xmax=5,Ymin=-1,Ymax=3]
  \DrawAxisGrids[Enlarge=2.5mm]{auto}{auto}
  \DefineCurve[Name=cf,Start=0.35,End=4.2,Trace]%
    <f>{0.6*cos(4.5*(x-4)+2.1)-1.2*sin(x-4)+0.1*x+0.2}
  \FindMax[Start=0.35,End=4.2]{f}[cf-max]
  \FindMax[Start=3,End=4]{f}[cf-maxlocal]
  \FindMin[Start=1,End=2]{f}[cf-minlocal]
  \MarkPts*[Color=red,Lines]{(cf-max)}
  \MarkPts*[Color=blue,Lines]{(cf-maxlocal)}
  \MarkPts*[Color=olive,Lines]{(cf-minlocal)}
  \GetXYcoord{(cf-max)}[\MyMaxX][\MyMaxY]
\end{GraphTikz}\\
The maximum is $M\approx\RoundNb{\MyMaxY}$, reached in $x\approx\RoundNb{\MyMaxX}$
```



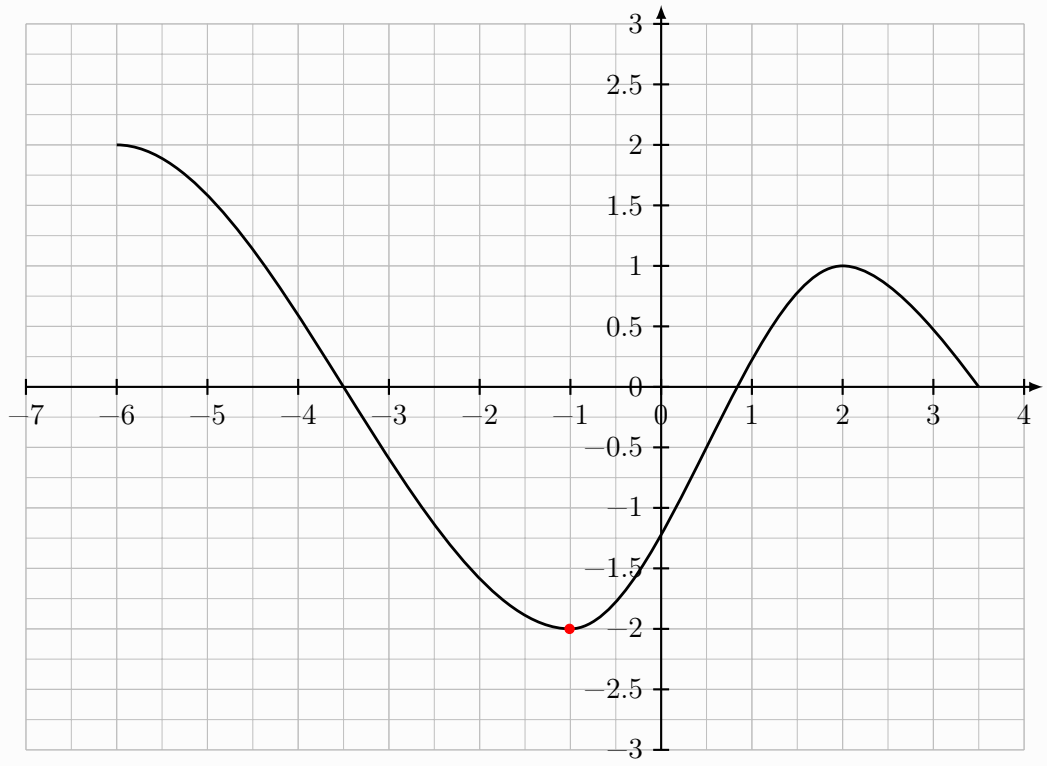
The maximum is $M \approx 2.17$, reached in $x \approx 2.17$

```
\begin{GraphTikz}[x=0.8cm,y=1cm,Xmin=-7,Xmax=4,Ymin=-3,Ymax=5]
  \DrawAxisGrids[Enlarge=2.5mm]{-7,-6,...,4}{-3,-2,...,5}
  \DefineInterpoCurve[Name=interpotest,Color=red,Trace,Tension=1]%
    {(-6,4)(-2,-2)(3,3.5)}
  \FindMin[Method=interpo,Tension=1]{(-6,4)(-2,-2)(3,3.5)}[interpo-min]
  \MarkPts*[Color=blue]{(interpo-min)}
  \GetXYcoord{(interpo-min)}[\MinInterpoX][\MinInterpoY]
\end{GraphTikz}\\
The minimum is $M\approx\RoundNb[3]{\MinInterpoY}$, reached at
↪  $x\approx\RoundNb[3]{\MinInterpoX}$
```



The minimum is $M \approx -2.003$, reached at $x \approx -1.908$

```
\begin{GraphTikz}%
  [x=1.2cm,y=1.6cm,Xmin=-7,Xmax=4,Ymin=-3,Ymax=3,Ygrid=0.5,Ygrids=0.25]
  \DrawAxisGrids[Enlarge=2.5mm]{auto}{auto}
  \def\LISTETEST{-6/2/0§-1/-2/0§2/1/0§3.5/0/-1}
  \DefineSplineCurve[Name=splinetest,Trace]{\LISTETEST}
  \FindMin[Method=spline]{\LISTETEST}[spline-min]
  \MarkPts*[Color=red]{(spline-min)}
\end{GraphTikz}
```



4.6 Integrals (improved version)

We can also work with integrals.
In this case it is preferable to highlight the domain **before** the plots, to avoid overprinting in relation to the curves/points.
It is possible to:

- represent an integral **under** a defined curve;
- represent an integral **between** two curves;
- the integration limits can be x-coordinates and/or nodes.

💣 Given the differences in processing between formula curves, simple interpolation curves or cubic interpolation curves, the arguments and keys may differ depending on the configuration!

```
%in the GraphiqueTikz environment
\DrawIntegral[keys]<specific options>{object1}[object2]{A}{B}
```

The optional `[keys]` for definition or tracing are:

- `Colors` =: colors of the filling, in the form `Col` or `ColBorder/ColBg` (`gray` by default);
- `Style`: type of filling, among `fill`/`hatch` (`fill` by default);
- `Opacity`: opacity (`0.5` by default) of the filling;
- `Hatch`: style (`north west lines` by default) of the hatch filling;
- `Type`: type of integral among
  - `fct` (default) for an integral under a curve defined by a formula;
  - `spl` for an integral under a curve defined by a cubic spline;
  - `itp` for an integral under a curve defined by interpolation;
  - `fct/fct` for an integral between two curves defined by a formula;
  - `fct/spl` for an integral between a curve (above) defined by a formula and a curve (below) defined by a spline cubic;
  - etc.
- `Step`: steps (calculated by default otherwise) for the plot;
- `Junction`: junction of segments (`bevel` by default);
- `Bounds`: type of terminals among:
  - `abs` for the limits given by the abscissa;
  - `nodes` for the limits given by the nodes;
  - `abs/node` for the limits given by abscissa and node;
  - `node/abs` for the limits given by node and abscissa;
- `Border`: boolean (`true` by default) to display the side lines,
- `SplineName`: macro (important!) of the spline generated previously for a higher version spline;
- `SplineNameB`: macro (important!) of the spline generated previously for a lower version spline;
- `InterpoName`: name (important!) of the interpolation curve generated previously, in higher version;

- `InterpoBName`: name (important!) of the interpolation curve generated previously, in lower version;
- `Tension`: Tension for the interpolation curve generated previously, in higher version;
- `TensionB`: Tension of the interpolation curve generated previously, in lower version.

The first required argument is the spline function or curve or list of interpolation points.

The next optional argument is the spline function or curve or list of interpolation points.

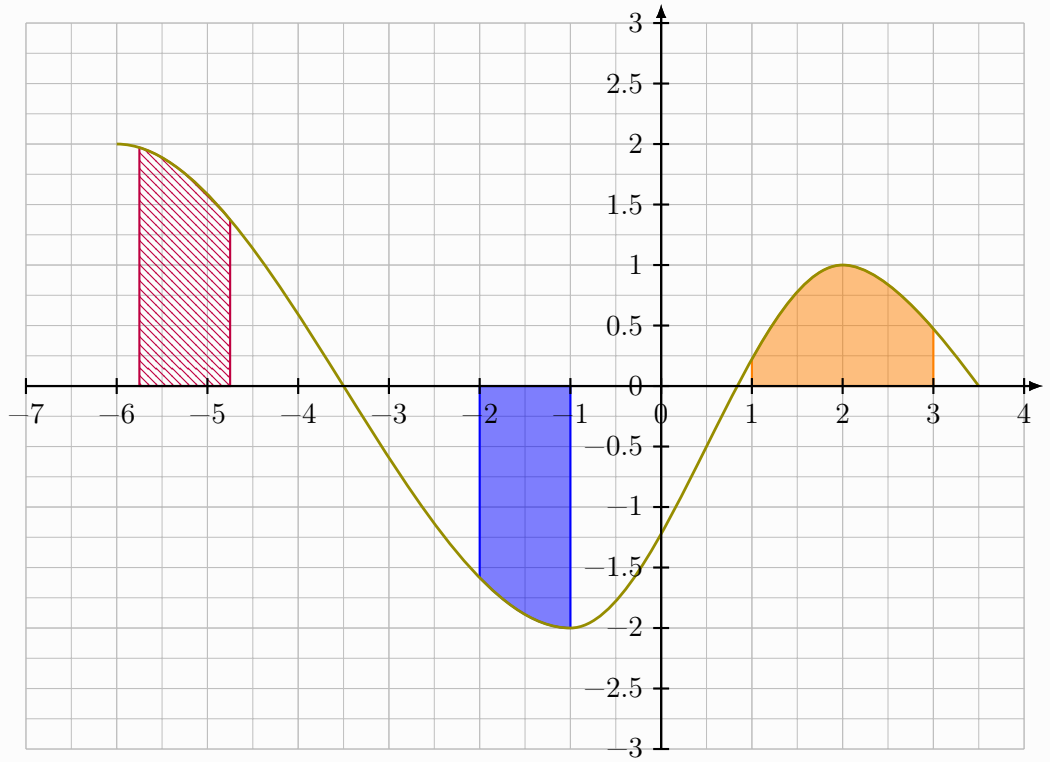
The last two mandatory arguments are the limits of the integral, given in a form consistent with the key `Bounds`.

In the case of curves defined by *points*, it is necessary to work on intervals on which the first curve is **above** the second.
It will undoubtedly be interesting to work with *intersections* in this case.

```
\begin{GraphTikz}%
  [x=0.6cm,y=0.06cm,
  Xmin=0,Xmax=21,Xgrid=1,Xgrids=0.5,
  Ymin=0,Ymax=155,Ygrid=10,Ygrids=5]
  \DrawAxisGrids[Grads=false,Enlarge=2.5mm]{}{}
  \DefineCurve[Name=cf,Start=1,End=20,Color=red]<f>{80*x*exp(-0.2*x)}
  \DrawIntegral
    [Bounds=abs,Colors=blue/cyan!50]%
   {f(x)}{3}{12}
  \DrawCurve[Color=red,Start=1,End=20]{f(x)}
  \DrawAxisGrids%
    [Grid=false,Enlarge=2.5mm,Font=\small]{0,1,...,20}{0,10,...,150}
\end{GraphTikz}
```



```
\begin{GraphTikz}%
  [x=1.2cm,y=1.6cm,Xmin=-7,Xmax=4,Ymin=-3,Ymax=3,Ygrid=0.5,Ygrids=0.25]
  \DrawAxisGrids[Grads=false,Enlarge=2.5mm]{}{}
  \def\LISTETEST{-6/2/0§-1/-2/0§2/1/0§3.5/0/-1}
  \DefineSplineCurve[Name=splinetest]{\LISTETEST}
  \DrawIntegral[Type=spl,Style=hatch,Colors=purple]{splinetest}{-5.75}{-4.75}
  \DrawIntegral[Type=spl,Colors=blue]{splinetest}{-2}{-1}
  \DrawIntegral[Type=spl,Colors=orange]{splinetest}{1}{3}
  \DrawSplineCurve[Color=olive]{\LISTETEST}
  \DrawAxisGrids[Grid=false,Enlarge=2.5mm]
    {-7,-6,...,4}%
    {-3,-2.5,...,3}
\end{GraphTikz}
```



4.7 Tangents

The idea of this command is to draw the tangent to a previously defined curve, specifying:

- the point (abscissa or node) at which we wish to work;
- possibly the direction (in the case of a discontinuity or a terminal);
- possibly the step (h) of the calculation;
- the *lateral spacings* to draw the tangent.

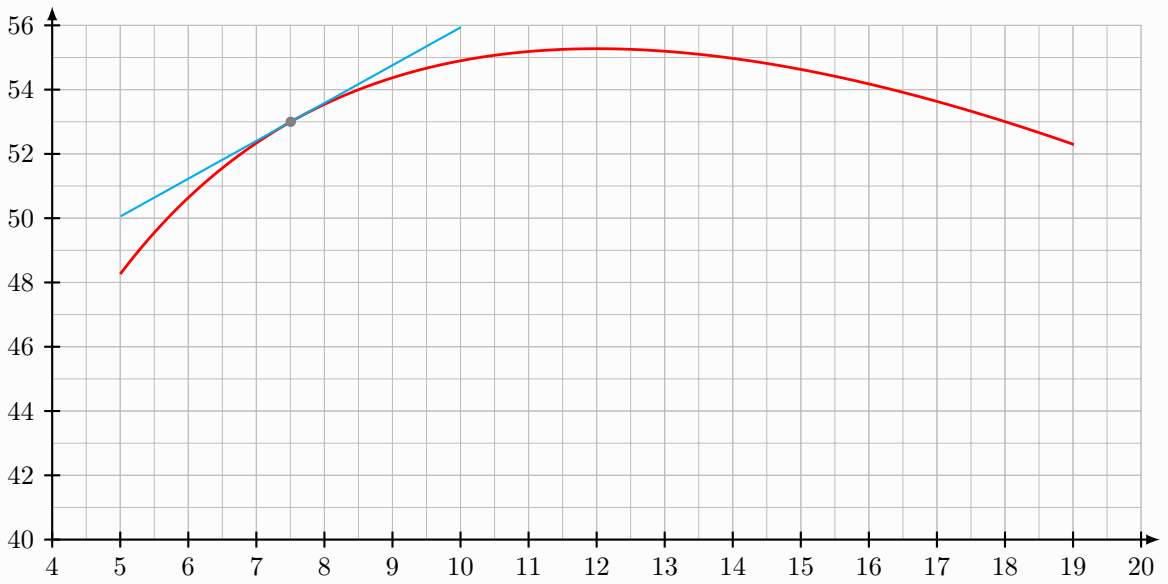
```
%in the GraphiqueTikz environment
\DrawTangent[keys]{function or curve}{point}<line options>
```

The optional `[keys]` for definition or tracing are:

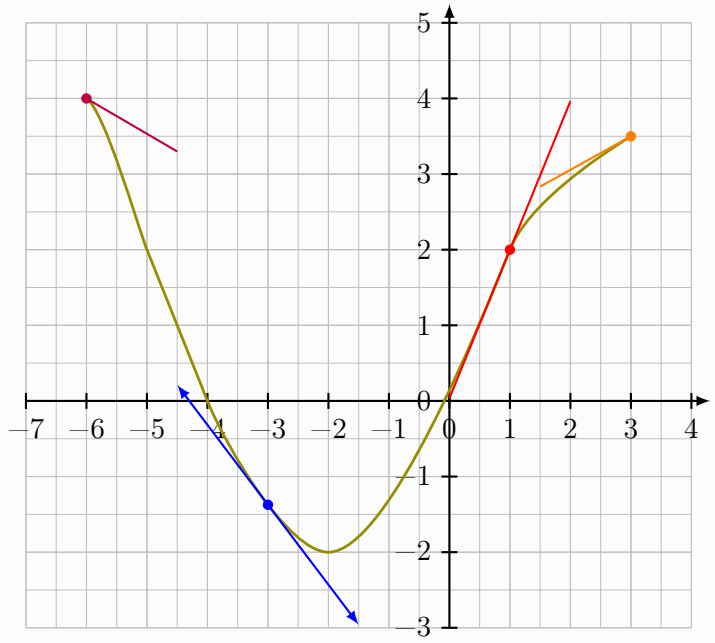
- `Colors` =: colors of the plots, in the form `Col` or `ColLine/ColPoint` (`black` by default);
- `OffsetL` =: left horizontal spacing to start the trace (`1` by default);
- `OffsetR` =: left horizontal spacing to start the trace (`1` by default);
- `DispPt`: boolean to display the support point (`false` by default);
- `Spline`: boolean to specify that a spline is used (`false` by default);
- `h`: delta h used for calculations (`0.01` by default);
- `Direction`: allows you to specify the *direction* of the tangent, among `lr`/`l`/`r` (`lr` by default);
- `Node`: boolean to specify that a node is used (`false` by default).

The first required argument is the spline function or curve (if applicable).
The last mandatory argument is the work point (abscissa version or node following the key `Node`).

```
\begin{GraphTikz}%
  [x=0.9cm,y=0.425cm,Xmin=4,Xmax=20,Origx=4,
  Ymin=40,Ymax=56,Ygrid=2,Ygrids=1,Origy=40]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{4,5,...,20}{40,42,...,56}
  \DefineCurve[Name=cf,Start=5,End=19,Color=red,Trace]<f>{-2*x+3+24*log(2*x)}
  \FindCounterimage[Color=teal,Name=JKL,Disp=false]{cf}{53}
  %tangent
  \DrawTangent%
    [Colors=cyan/gray,OffsetL=2.5,OffsetR=2.5,Node,DispPt]{f}{(JKL-1)}
\end{GraphTikz}
```



```
\begin{GraphTikz}%
  [x=0.8cm,y=1cm,Xmin=-7,Xmax=4,Ymin=-3,Ymax=5]
  \DrawAxisGrids[Enlarge=2.5mm]{-7,-6,...,4}{-3,-2,...,5}
  \def\LISTETEST{-6/4/-0.5§-5/2/-2§-4/0/-2§-2/-2/0§1/2/2§3/3.5/0.5}
  \DefineSplineCurve[Name=splinetest,Trace,Color=olive]{\LISTETEST}
  \DrawTangent[Colors=red,Spline,DispPt]{splinetest}{1}
  \DrawTangent%
  [Colors=blue,Spline,OffsetL=1.5,OffsetR=1.5,DispPt]{splinetest}{-3}%
    <tkzgrpharrowlr>
  \DrawTangent[Direction=l,Colors=orange,Spline,OffsetL=1.5,DispPt]{splinetest}{3}
  \DrawTangent[Direction=r,Colors=purple,Spline,OffsetR=1.5,DispPt]{splinetest}{-6}
\end{GraphTikz}
```



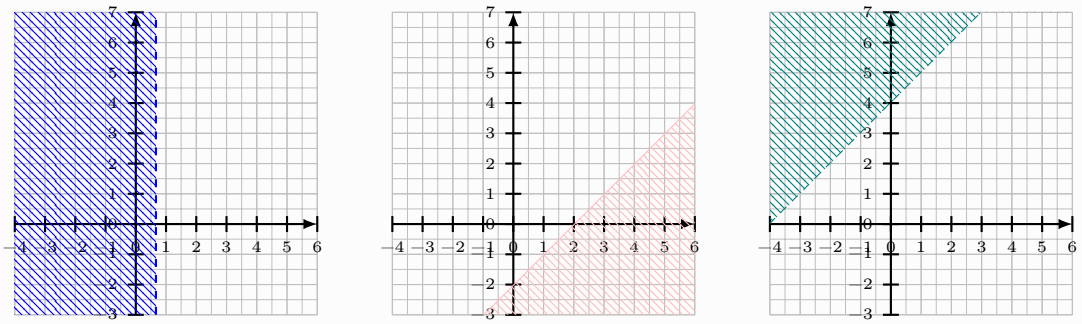
4.8 Linear inequality

```
%within graphTike environment
\LinearInequality[keys]<function name>{expression}{symbol}
```

First argument, *optional* and within `[...]`, gives following keys:

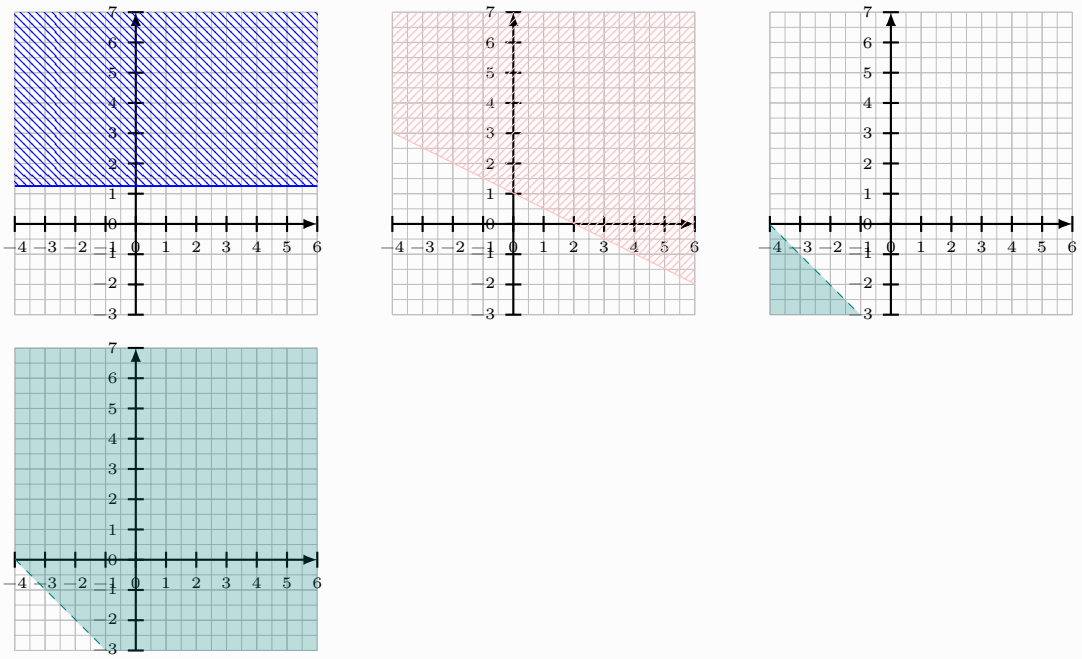
- `swap=TF` for shading non-solution (`false` by default) ;
- `opacity=...` (`0.25` by default) ;
- `color=...` (`black` by default) ;
- `style=...` (`hatch` by default) ;
- `hatch` (`north west lines` by default).

```
\begin{GraphTikz}[x=0.4cm,y=0.4cm,Xmin=-4,Xmax=6,Ymin=-3,Ymax=7]
  \DrawAxisGrids[Font=\tiny]{auto}{auto}
  \LinearInequality[color=blue]{-3x+2}{>0}
\end{GraphTikz}
~~
\begin{GraphTikz}[x=0.4cm,y=0.4cm,Xmin=-4,Xmax=6,Ymin=-3,Ymax=7]
  \DrawAxisGrids[Font=\tiny]{auto}{auto}
  \LinearInequality[color=pink]{-x+y+2}{<=0}
\end{GraphTikz}
~~
\begin{GraphTikz}[x=0.4cm,y=0.4cm,Xmin=-4,Xmax=6,Ymin=-3,Ymax=7]
  \DrawAxisGrids[Font=\tiny]{auto}{auto}
  \LinearInequality[color=teal]{-x+y-4}{>0}
\end{GraphTikz}
```



```
\begin{GraphTikz}[x=0.4cm,y=0.4cm,Xmin=-4,Xmax=6,Ymin=-3,Ymax=7]
  \DrawAxisGrids[Font=\tiny]{auto}{auto}
  \LinearInequality[color=blue]{-4*y+5}{<=0}
\end{GraphTikz}
~~
\begin{GraphTikz}[x=0.4cm,y=0.4cm,Xmin=-4,Xmax=6,Ymin=-3,Ymax=7]
  \DrawAxisGrids[Font=\tiny]{auto}{auto}
  \LinearInequality[color=pink,hatch={north east lines}]{-x-2y+2}{<=0}
\end{GraphTikz}
~~
\begin{GraphTikz}[x=0.4cm,y=0.4cm,Xmin=-4,Xmax=6,Ymin=-3,Ymax=7]
  \DrawAxisGrids[Font=\tiny]{auto}{auto}
  \LinearInequality[style=fill,color=teal]{-x-y-4}{>0}
\end{GraphTikz}

\begin{GraphTikz}[x=0.4cm,y=0.4cm,Xmin=-4,Xmax=6,Ymin=-3,Ymax=7]
  \DrawAxisGrids[Font=\tiny]{auto}{auto}
  \LinearInequality[swap,style=fill,color=teal]{-x-y-4}{>0}
\end{GraphTikz}
```



5 Commands specific to two-variable statistics

5.1 Limitations

Given the specific features of Ti*k*Z, we advise you not to use values that are too *large* at axis level (this can cause problems with years, for example), or else you'll have to *transform* axis and/or data values so that everything is displayed as it should be (also beware of regressions, calculations, etc.).

5.2 The point scatter

In addition to commands linked to functions, it is also possible to represent double statistical series.
The following paragraph shows that adding a key allows you to add the linear adjustment line.

```
%in the GraphiqueTikz environment
\DrawScatter[keys]{ListX}{ListY}
```

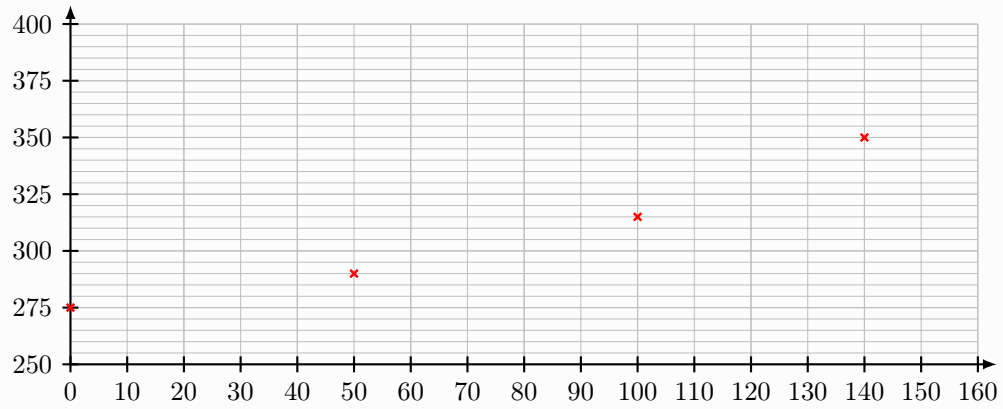
The optional `[key]` is:

- `ColorScatter`: color of the cloud points (`black` by default).

The mandatory arguments allow you to specify:

- the list of x;
- the list of y.

```
\begin{GraphTikz}%
  [x=0.075cm,y=0.03cm,Xmin=0,Xmax=160,Xgrid=20,Xgrids=10,
  Origy=250,Ymin=250,Ymax=400,Ygrid=25,Ygrids=5]
  %window preparation
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{0,10,...,160}{250,275,...,400}
  %A cloud of dots
  \DrawScatter[Style=x,ColorScatter=red]{0,50,100,140}{275,290,315,350}
\end{GraphTikz}
```



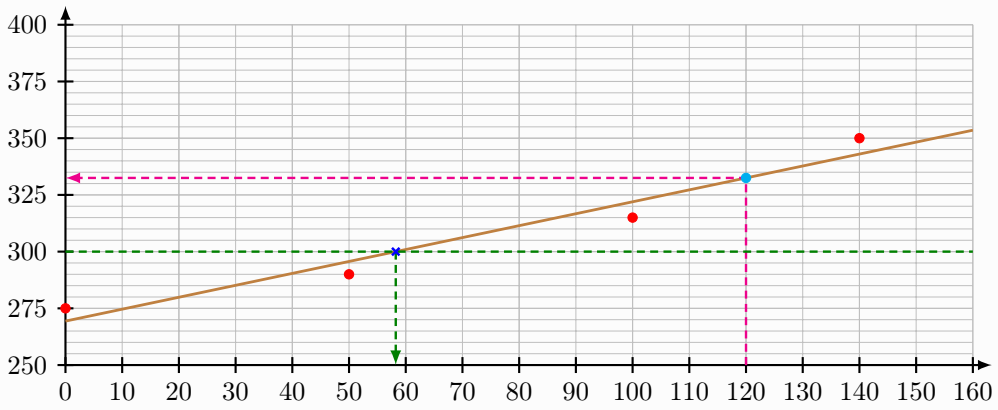
5.3 The regression line

The linear regression line (obtained by the least squares method) can easily be added, using the key `DrawLine`.
In this case, new keys are available:

- `ColorLine`: color of the line (`black` by default);

- `Rounds`: precision of coefficients (`empty` by default);
- `Start`: initial abscissa of the plot (`\pflxmin` by default);
- `End`: terminal abscissa of the plot (`\pflxmax` by default);
- `Name`: name of the line, for later use (`reglin` by default).

```
\begin{GraphTikz}%
  [x=0.075cm,y=0.03cm,Xmin=0,Xmax=160,Xgrid=20,Xgrids=10,
  Origy=250,Ymin=250,Ymax=400,Ygrid=25,Ygrids=5]
  \DrawAxisGrids[Enlarge=2.5mm,Font=\small]{0,10,...,160}{250,275,...,400}
  %cloud and right
  \DrawScatter%
    [ColorScatter=red,ColorLine=brown,DrawLine]%
  {0,50,100,140}{275,290,315,350}
  %picture
  \DrawRanges[Colors=cyan/magenta,Lines]{d}{120}
  %history
  \DrawCounterimage[Style=x,Colors=blue/green!50!black,Lines]{reglin}{300}
\end{GraphTikz}
```



5.4 Other regressions

In partnership with the `xint-regression` package, loaded by the package (but *can be deactivated* via the `[noxintreg]` option), it is possible to work on other types of regression:

- linear $\boxed{ax+b}$;
- quadratic $\boxed{ax^2+bx+c}$;
- cubic $\boxed{ax^3+bx^2+cx+d}$;
- power $\boxed{ax^b}$;
- exponential $\boxed{ab^x}$ or $\boxed{e^{ax+b}}$ or $\boxed{be^{ax}}$ or $\boxed{C+be^{ax}}$;
- logarithmic $\boxed{a+b\ln(x)}$;
- hyperbolic $\boxed{a+\dfrac{b}{x}}$.

The command, similar to that of defining a curve, is:

```
\DrawRegression[keys]<name fct>{type}<rounded>{listex}{listey}
```

The `[keys]` available are, classically:

- `Start`: lower bound of the definition set (`\pflxmin` by default);
- `End`: lower bound of the definition set (`\pflxmax` by default);
- `Name`: name of the curve (important for the rest!);
- `Color`: color of the trace (`black` by default);
- `Step`: plot step (it is determined *automatically* at the start but can be modified).

The second argument, optional and between `<...>`, allows you to name the regression function.
The third argument, mandatory and between `{...}` allows you to choose the type of regression, among:

- `lin`: linear $\boxed{ax+b}$;
- `quad`: quadratic $\boxed{ax^2+bx+c}$;
- `cub`: cubic $\boxed{ax^3+bx^2+cx+d}$;
- `pow`: power $\boxed{ax^b}$;
- `expab`: exponential $\boxed{ab^x}$
- `hyp`: hyperbolic $\boxed{a+\dfrac{b}{x}}$;
- `log`: logarithmic $\boxed{a+b\ln(x)}$;
- `exp`: exponential $\boxed{e^{ax+b}}$;
- `expalt`: exponential $\boxed{be^{ax}}$;
- `expoff=C`: exponential $\boxed{C+be^{ax}}$.

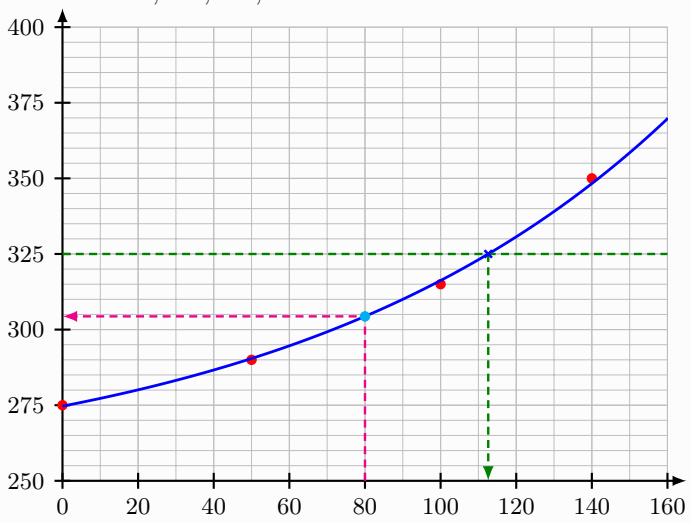
The fourth argument, optional and between `<...>`, allows you to specify the rounding(s) for the coefficients of the regression function.
The last two arguments are the lists of values of X and Y.

```
\def\LISTEXX{0,50,100,140}\def\LISTEYY{275,290,315,350}%
ListX:= \LISTEXX\\
ListY:= \LISTEYY

\begin{GraphTikz}
  [x=0.05cm,y=0.04cm,Xmin=0,Xmax=160,Xgrid=20,Xgrids=10,
  Origy=250,Ymin=250,Ymax=400,Ygrid=25,Ygrids=5]
  %window preparation
  \DrawAxisGrids[Enlarge=2.5mm,Font=\footnotesize]{auto}{auto}
  %A cloud of dots
  \DrawScatter[Style=o,ColorScatter=red]{\LISTEXX}{\LISTEYY}
  %adjustment expoffset
  \DrawRegression[Color=blue,Name=adjust]<adjust>{expoff=250}{\LISTEXX}{\LISTEYY}
  %holdings
  \DrawRanges[Colors=cyan/magenta,Lines]{adjust}{80}
  \DrawCounterimage[Style=x,Colors=blue/green!50!black,Lines]{adjust}{325}
\end{GraphTikz}

\xintexpoffreg[offset=250,round=3/1]{\LISTEXX}{\LISTEYY}%
We obtain $y=250+\num{\expregoffb}\text{e}^{\num{\expregoffa}x}$
```

ListX:= 0,50,100,140
ListY:= 275,290,315,350



We obtain $y = 250 + 24.7\text{e}^{0.01x}$

6 Auxiliary commands

6.1 Intro

In addition to purely *graphic* commands, some auxiliary commands are available:

- a to format a number with a given precision;
- one for working on random numbers, with constraints.

6.2 Formatted rounding

The `\RoundNb` command allows you to format, using the `siunitx` package, a number (or a calculation), with a given precision. This can be *useful* for formatting results obtained using coordinate retrieval commands, for example.

```
\RoundNb[precision]{xint calculation}
```

```
\RoundNb{1/3}\\
\RoundNb{16.1}\\
\RoundNb[3]{log(10)}\\
```

0.33
16.1
2.303

6.3 Random number under constraints

The idea of this second command is to be able to determine a random number:

- integer or decimal;
- under constraints (between two fixed values).

This can allow, for example, to work on curves with *random* points, but respecting certain constraints.

```
\PickRandomNb(*)[precision (def 0)]{lower limit}{upper limit}[\macro]
```

The star version takes the constraints in strict form (lower bound $<$ macro $<$ upper bound) while the normal version takes the constraints in broad form (lower bound) $\leq$ macro $\leq$ upper bound).

Note that the *terminals* can be existing *macros*!

```
%a number (2 digits after the decimal point) between 0.75 and 0.95
%a number (2 digits after the decimal point) between 0.05 and 0.25
%a number (2 decimal places) between 0.55 and \YrandMax
%a number (2 decimal places) between \YrandMin and 0.45
\PickRandomNb[2]{0.75}{0.95}[\YrandMax]%
\PickRandomNb[2]{0.05}{0.25}[\YrandMin]%
\PickRandomNb*[2]{0.55}{\YrandMax}[\YrandA]%
\PickRandomNb*[2]{\YrandMin}{0.45}[\YrandB]%
%verification
\num{\YrandMax} \& \num{\YrandMin} \& \num{\YrandA} \& \num{\YrandB}
```

0.82 & 0.19 & 0.74 & 0.23

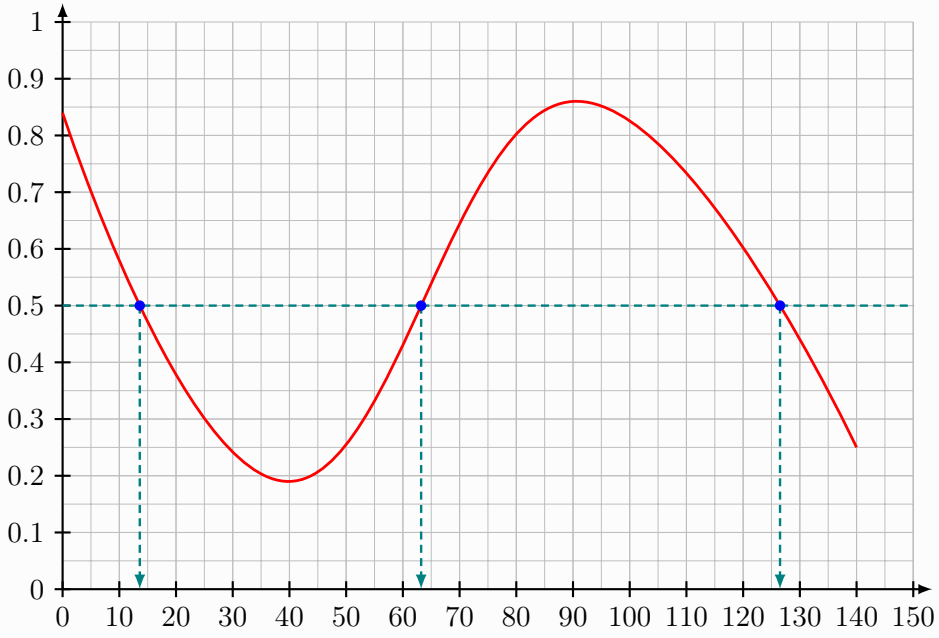
```
%a number (2 digits after the decimal point) between 0.75 and 0.95
%a number (2 digits after the decimal point) between 0.05 and 0.25
%a number (2 decimal places) between 0.55 and \YrandMax
%a number (2 decimal places) between \YrandMin and 0.45
\PickRandomNb[2]{0.75}{0.95}[\YrandMax]%
\PickRandomNb[2]{0.05}{0.25}[\YrandMin]%
\PickRandomNb*[2]{0.55}{\YrandMax}[\YrandA]%
\PickRandomNb*[2]{\YrandMin}{0.45}[\YrandB]%
%verification
\num{\YrandMax} \& \num{\YrandMin} \& \num{\YrandA} \& \num{\YrandB}
```

0.95 & 0.15 & 0.84 & 0.27

```
%the curve is designed so that there are 3 antecedents
\PickRandomNb[2]{0.75}{0.95}[\YrandMax]%
\PickRandomNb[2]{0.05}{0.25}[\YrandMin]%
\PickRandomNb*[2]{0.55}{\YrandMax}[\YrandA]%
\PickRandomNb*[2]{\YrandMin}{0.45}[\YrandB]%

\begin{GraphTikz}
  [x=0.075cm,y=7.5cm,Xmin=0,Xmax=150,Xgrid=10,Xgrids=5,
  Ymin=0,Ymax=1,Ygrid=0.1,Ygrids=0.05]
  \DrawAxisGrids[Last,Enlarge=2.5mm]{auto}{auto}
  \DefineInterpoCurve[Color=red,Trace,Name=functiontest,Tension=0.75]
    {(0,\YrandA)(40,\YrandMin)(90,\YrandMax)(140,\YrandB)}
  \FindCounterimage[Disp=false,Name=ANTECED]{functiontest}{0.5}
  \DrawCounterimage[Colors=blue/teal,Lines]{functiontest}{0.5}
  \GetXcoord{(ANTECED-1)}[\Aalpha]
  \GetXcoord{(ANTECED-2)}[\Bbeta]
  \GetXcoord
  {(ANTECED-3)}[\Cgamma]
\end{GraphTikz}

The solutions of $f(x)=\num{0.5}$ are, by graphic reading:
$\begin{cases}
  \alpha \approx \RoundNb[0]{\Aalpha} \\
  \beta \approx \RoundNb[0]{\Bbeta} \\
  \gamma \approx \RoundNb[0]{\Cgamma}
\end{cases}$.
```



The solutions of $f(x) = 0.5$ are, by graphic reading: $\begin{cases} \alpha \approx 14 \\ \beta \approx 63 \\ \gamma \approx 127 \end{cases}$.

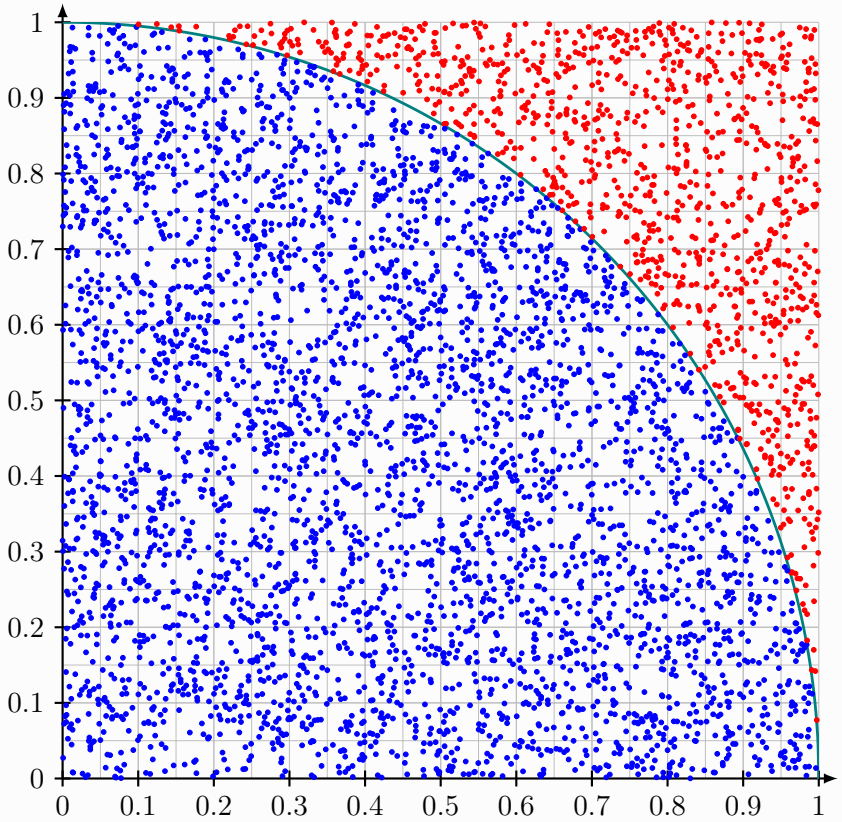
6.4 Monte-Carle method

```
%in the GraphiqueTikz environment
\SimulateMonteCarlo[keys]{<function>}{number of points}[\nbptsmcok][\nbptsmcko]
```

```
\begin{GraphTikz}%
  [x=10cm,y=10cm,Xmin=0,Xmax=1,Xgrid=0.1,Xgrids=0.05,
  Ymin=0,Ymax=1,Ygrid=0.1,Ygrids=0.05]
  \DrawAxisGrids[Enlarge=2.5mm,Last]{auto}{auto}
  \DefineCurve[Trace,Color=teal,Step=0.001]<f>{sqrt(1-x^2)}
  \SimulateMonteCarlo<f>{5000}
\end{GraphTikz}

There is \textcolor{blue}{\num{\nbptsmcok}} blue points,
there is \textcolor{red}{\num{\nbptsmcko}} red points.

And $\frac{\num{\nbptsmcok}}{\num{\nbptsmc}}
\approx \RoundNb[4]{\nbptsmcok/\nbptsmc}$
et $\frac{\pi}{4} \approx \RoundNb[4]{pi/4}$.
```



There is 3927 blue points, there is 1073 red points.
And $\frac{3927}{5000} \approx 0.7854$ et $\frac{\pi}{4} \approx 0.7854$.

6.5 Some pgfplots macros

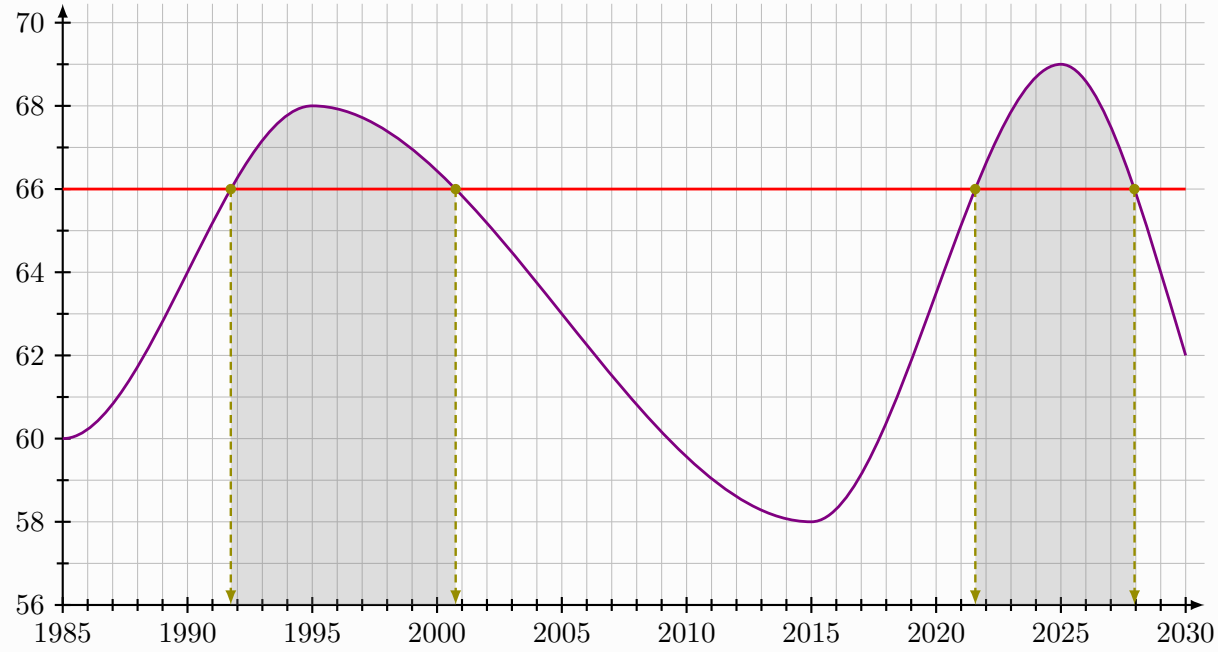
In addition to pgfplots/axis, there's few *simple* macros in order to work with `pgfplots/axis` environment.

```
%find intersection of two [name path] objects defined
\findintersectionspgf[nodename baises]{object1}{object2}[\myt]
%global extraction of coordinates
\gextractxnodepgf{node}[\myxcoord]
\gextractynodepgf{node}[\myycoord]
\gextractxynodepgf{node}[\myxcoord][\myycoord]
%area between curves
\fillbetweencurvespgf[tikz options]{curve1}{curve2}<soft domain options>
%cubic splines
\addplotspline(*)[tikz options]<coeffs>{list of points}[\myspline]
```

```
%\usepackage{alphalph}

\begin{tikzpicture}
  \begin{axis}%
    [%
    axis y line=center,axis x line=middle,                                  %axis
    axis line style={line width=0.8pt,-latex},
    x=0.33cm,y=0.55cm,xmin=1985,xmax=2030,ymin=56,ymax=70,                  %units
    grid=both,xtick distance=5,ytick distance=2,                            %gridp
    minor x tick num=4,minor y tick num=1,                                  %grids
    extra x ticks={1985},extra x tick style={grid=none},                    %origx
    extra y ticks={56},extra y tick style={grid=none},                      %origy
    x tick label style={/pgf/number format/.cd,use comma,1000 sep={}},      %year
    major tick length={2*3pt},minor tick length={1.5*3pt},                  %grads
    every tick/.style={line width=0.8pt},enlargelimits=false,               %style
    enlarge x limits={abs=2.5mm,upper},enlarge y limits={abs=2.5mm,upper},  %énlarge
    ]
    %spline + y=66
    \addplot[name path global=eqtest,mark=none,red,line width=1.05pt,domain=1985:2030]
    ↪  {66} ;
    \def\LISTETEST{1985/60/0§1995/68/0§2015/58/0§2025/69/0§2030/62/-2}
    \addplotspline*[line width=1.05pt,violet,name path
    ↪  global=splinecubtest]{\LISTETEST}[\monsplineviolet]
    %equation f(x)=66
    \findintersectionspgf[MonItsc]{eqtest}{splinecubtest}
    %extraction of coordinates
    \gextractxnodepgf{(MonItsc-1)}[\xMonItscA]
    \gextractxnodepgf{(MonItsc-2)}[\xMonItscB]
    \gextractxnodepgf{(MonItsc-3)}[\xMonItscC]
    \gextractxnodepgf{(MonItsc-4)}[\xMonItscD]
    %vizualisation
    \xintFor* #1 in {\xintSeq{1}{4}}\do{%
      \draw[line width=0.9pt,densely dashed,olive,->,>=latex] (MonItsc-#1) -- (\csname
      ↪  xMonItsc\AlphAlph{#1}\endcsname,56) ;
      \filldraw[olive] (MonItsc-#1) circle[radius=1.75pt] ;
    }
    %area
    \path [name path=xaxis] (1985,56) -- (2030,56);
    \fillbetweencurvespgf{splinecubtest}{xaxis}<domain={\xMonItscB:\xMonItscA}>
    \fillbetweencurvespgf{splinecubtest}{xaxis}<domain={\xMonItscD:\xMonItscC}>
  \end{axis}
\end{tikzpicture}

Solutions of  $f(x)=66$ are \RoundNb[0]{\xMonItscA} \&\ \RoundNb[0]{\xMonItscB} \&\
↪  \RoundNb[0]{\xMonItscC} \&\ \RoundNb[0]{\xMonItscD}.
```



Solutions of $f(x) = 66$ are 1992 & 2001 & 2022 & 2028.

7 History

```
0.32b: Improvments in [alt/normalize] version
0.31f: Improvments in [alt/normalize] version
0.31e: Improvments in [fr] version
0.31d: Improvments in [fr] version (nice numbers, minigraph, sequences)
0.31c: Improvments in [fr] version
0.31b: Improvments in [fr] and [alt] version
0.30f: Bugfix + new commands (stats in [fr] version)
0.30e: New commands (experimental in [fr] version)
0.30d: New commands (experimental in [fr] version)
0.30c: New version of code, with [en/fr] separate versions + macros for calculate
------ New commands (experimental in [fr] version)
0.30b: New commands (experimental in [fr] version)
0.30a: Nodes for windows and axis
0.2.9: New keys for french version (work in progress)
0.2.7: Ability to specify dimensions for environment (test)
0.2.6: Linear inequality (work in progress)
0.2.5: Lagrange interpolation ([french] for the moment) + enhancements
0.2.4: Bugfix with [fr] version
0.2.3: Bugfix with a length
0.2.2: Bugfixes
0.2.0: [Alt] key for Hermite spline + few pgfplots macros
0.1.9: Bugfix
0.1.8: New commands [in french doc] (binomial, cabweb,\ldots)
0.1.6: Vertical asymptote + [in french doc] commands for integrals
0.1.5: Initial version [en]
```